Syrian private university

Faculty of Engineering



AQARI WEB

مشروع تخرج2

اعداد الطلاب :

محمد محمد راغب عبيد

يوسف محمد الرفاعي

بإشراف المهندس:

ماهر صارم

2026

أهداء

بِقَلْبٍ مُمتَنٍّ، نُهدي هذا النّجاح، الذي لم يكن ليكتملَ لولا توجيهاتُك، إلى التي كانت لنا سنداً منذ البداية

المهندس ماهر صارم

إلى الذي لولاهُ، لما كانت كليّتنا مميّزةً بكادرِها ومِنهاجِها. إلى الذي يَبذل كلّ جُهدِه كي نكون الأميزَ والأفضل

الدكتور مهيب النقري

المستخلص

يهدف هذا المشروع إلى تصميم وتطوير منصة عقارية متكاملة باسم «عقاري» تخدم السوق العقاري في الجمهورية العربية السورية، من خلال توفير بيئة رقمية تجمع المالكين والمستأجرين والمشترين في مكان واحد. اعتمد المشروع على منهجية التطوير الرشيقة (Agile) ضمن إطار Scrum، وبُني باستخدام إطار العمل Django بلغة Python، وقاعدة بيانات SQLite، وتقنيات الويب الحديثة. يدعم النظام المحافظات السورية الأربع عشرة، وتصنيف العقارات حسب النوع والحالة، مع نظام تقييم حسابي، وآلية لعرض العقارات الأكثر مشاهدة وفق بيانات حقيقية، ولوحة إدارة متطورة. أثبتت نتائج الاختبار تحقيق جميع الأهداف بنسبة نجاح 100% ورضى مستخدمين تجاوز 92%.

الكلمات المفتاحية: منصة عقارية، Python ،Django، تطوير الويب، السوق السوري، Agile، نمط MVT، قواعد البيانات.

Abstract

This project aims to design and develop an integrated real estate platform named "Aqari" to serve the real estate market in the Syrian Arab Republic. The platform provides a digital environment that brings together property owners, tenants, and buyers in one place. The Syrian real estate market faces numerous challenges, including difficulty accessing reliable information and the absence of modern electronic systems for managing property listings.

The project adopted the Agile development methodology within the Scrum framework. The system was built using the Django framework with Python as the backend, an SQLite database, and modern web technologies (HTML5, CSS3, Bootstrap 5, JavaScript). The system supports all fourteen Syrian governorates, with property categorization by type (apartments, villas, lands, offices) and status (for sale, for rent).

The system includes several intelligent features, most notably an arithmetic rating system for properties based on user opinions, a ranking mechanism for the most-viewed properties based on real data, an advanced multi-criteria search system, favorites management, and an advanced administrative dashboard with a modern design. The interface is fully bilingual with RTL support and a responsive design for all devices.

Test results proved the effectiveness of the system in achieving its objectives, with a property display accuracy of 100%, a response time of less than two seconds in most operations, and a user satisfaction rate exceeding 92% in preliminary tests.

Keywords: Real Estate Platform, Django, Python, Web Development, Syrian Market, Recommendation Systems, Agile, MVT Pattern, Relational Databases.

Table of Contents

SECTION ONE: PROJECT INTRODUCTION

Chapter 1: General Introduction

Chapter 2: Project Development & Management Methodology

SECTION TWO: THEORETICAL STUDY & ANALYSIS

Chapter 3: Literature Review & Previous Studies

Chapter 4: Requirements Modeling & Analysis

SECTION THREE: DESIGN & ARCHITECTURE

Chapter 5: System Architectural Design

SECTION FOUR: IMPLEMENTATION & TESTING

Chapter 6: Implementation Environment & Tools

Chapter 7: Software Implementation

Chapter 8: Testing & Evaluation

SECTION FIVE: RESULTS & RECOMMENDATIONS

Chapter 9: Results & Analysis

Chapter 10: Conclusion & Recommendations

SECTION SIX: APPENDICES & REFERENCES

Appendices

References

List of Tables

Table 2-1: Comparison between Agile and Waterfall

Table 2-2: Project Timeline (Sprints)

Table 2-3: Risk Management Register

Table 4-1: Stakeholder Analysis

Table 4-2: Functional Requirements (25 FR)

Table 4-3: Non-Functional Requirements

Table 4-4: MoSCoW Prioritization

Table 4-5: Use Case Descriptions

Table 4-6: Requirements Traceability Matrix

Table 5-1: Property Table Schema

Table 5-2: Database Constraints

Table 8-1: Test Cases

Table 8-2: Test Results Summary

Table 9-1: Comparison with Existing Systems

List of Figures

Figure 4-1: Use Case Diagram

Figure 4-2: Activity Diagram (Add Property)

Figure 4-3: Sequence Diagram (Rate Property)

Figure 5-1: System Architecture (MVT Pattern)

Figure 5-2: Entity Relationship Diagram (ERD)

List of Abbreviations

Term	Definition
Django	Open-source Python web framework using the MVT pattern
MVT	Model-View-Template architectural pattern used by Django
ORM	Object Relational Mapping technique
CRUD	Create, Read, Update, Delete database operations
RTL	Right-To-Left text direction (for Arabic)
SQLite	Lightweight file-based relational database system
Bootstrap	CSS framework for responsive web interfaces
REST	Representational State Transfer architectural style
UI/UX	User Interface / User Experience
CSRF	Cross-Site Request Forgery (a web attack)
SCM	Software Configuration Management
Agile	Iterative software development methodology
Scrum	A framework within the Agile methodology
MoSCoW	Requirements prioritization (Must, Should, Could, Won't)

SECTION ONE

Project Introduction

Chapter 1

General Introduction

Objective: To introduce the project to the reader, explain its importance, and highlight what is new about it.

1.1 Project Overview and Background

The "Aqari" project falls within the field of management information systems and intelligent web applications, which are vital areas that have witnessed remarkable development in recent years at both global and regional levels. The need for specialized digital platforms in the real estate sector is increasing due to the technological transformation the world is experiencing today.

In the Syrian Arab Republic, particularly after years of economic and social transformations, the real estate sector emerges as one of the most vital sectors in need of modern digital solutions. Traditional methods of marketing and displaying properties still prevail, where owners and tenants resort to real estate offices, newspaper advertisements, or non-specialized social media platforms, leading to lost opportunities and difficulty accessing reliable data.

The "Aqari" system provides an integrated web platform that allows users to browse properties for sale or rent in all Syrian governorates, add their own listings, and interact with them through favorites, ratings, or reporting, in addition to an advanced management system that helps administrators monitor and verify listings.

To appreciate the significance of this project, it is helpful to consider the broader trajectory of digital transformation in the real estate industry. Globally, the sector has moved through several distinct phases. The first phase was simple online classified advertisements, which merely digitized the newspaper listing without adding any intelligence. The second phase introduced searchable databases with structured filters, allowing seekers to narrow results by criteria such as location, price, and type. The third and current phase incorporates data-driven intelligence: aggregate ratings, popularity signals derived from real user behavior, personalized recommendations, and analytics. Many regional markets, however, and the Syrian market in particular, remain largely stranded in the first phase, relying on unstructured social media posts and word of mouth. The Aqari project represents a deliberate effort to bring the Syrian market directly into the third phase.

This leap is not merely a matter of convenience. The absence of a structured, intelligent platform imposes real economic costs on all participants. Property owners struggle to reach the right audience and often accept suboptimal terms because they cannot effectively advertise. Seekers waste significant time and travel pursuing listings that prove unsuitable because the available information was incomplete or unreliable. The market as a whole operates with high friction and low transparency, which suppresses transaction volume and distorts pricing. A well-designed digital platform reduces this friction systematically, and the value it creates is therefore not incremental but structural.

The choice to frame this as an intelligent information system, rather than a mere website, is central to its academic positioning. The intelligence lies not in any single advanced algorithm but in the systematic transformation of raw user actions into useful aggregate knowledge: individual ratings become a community quality signal, individual page views become a popularity ranking, and individual listings become a searchable, comparable market. This is the essence of what an information system does — it converts data into decision-relevant information — and demonstrating this transformation on a locally relevant problem is the intellectual core of the project.

1.2 Problem Statement and Practical/Academic Importance

The Syrian real estate market suffers from several problems that can be summarized as follows:

- Absence of a unified and reliable Arabic real estate platform specifically serving the Syrian market.
- Difficulty accessing complete information about a property before a field visit, wasting time for both owners and seekers.
- Reliance on real estate brokers who may have bias or interest in directing customers to specific properties.
- Difficulty comparing properties and prices across different regions quickly and effectively.
- Absence of a property rating system through which users can benefit from others' experiences.

Hence, the practical importance of the project lies in providing a comprehensive digital solution to these problems. The academic importance lies in applying the theoretical concepts studied during the years of study, such as systems analysis and design, databases, object-oriented programming, user interface design, and software project management.

1.3 Project Objectives (Main and Sub)

Main Objective:

To build an integrated real estate web platform named "Aqari" that serves the real estate market in the Syrian Arab Republic, combining ease of use, professionalism, and efficiency, following the SMART methodology (Specific, Measurable, Achievable, Realistic, Time-bound).

Sub-Objectives:

1. Provide a professional Arabic user interface that supports RTL direction and works on all devices.
2. Enable users to create personal accounts and add their properties easily.
3. Provide an advanced multi-criteria search system (governorate, type, status, price).
4. Develop an arithmetic rating system for properties based on user opinions.
5. Build an intelligent mechanism to display the most-viewed properties based on real data.
6. Develop an advanced administrative dashboard with a modern design.
7. Ensure a response time not exceeding two seconds for any main operation.
8. Achieve 100% accuracy in data display and storage.

1.4 Project Scope, Constraints, and Assumptions

Scope:

The system includes an integrated web platform serving the Syrian real estate sector with its various features: user management, property management, search and filtering, rating, favorites, reporting system, and an admin dashboard.

Constraints:

- The time allotted to complete the project does not exceed one academic year.
- The limited budget requires the use of free open-source technologies.
- Lack of extensive real data on Syrian properties, which required the use of representative test data.
- The system is limited in the first phase to the web interface without native mobile applications.

Assumptions:

- The system assumes that users have an internet connection and basic knowledge of using a computer or smartphone.
- It is assumed that users will act honestly and responsibly when publishing their listings.
- It is assumed that the hosting server can handle a moderate number of concurrent visitors.

1.5 Project Contribution to Knowledge and Field

The "Aqari" project provides multiple contributions to the field of information systems and Arabic web applications:

9. The first web real estate platform fully dedicated to the Syrian market at this level of integration, combining 14 Syrian governorates in one system.
10. Integration of an intelligent arithmetic rating system with a mechanism for displaying the most-viewed properties based on real, not hypothetical, data.
11. Design of a professional Arabic admin dashboard with a modern Material Design, reusable as a template for similar projects.
12. Practical application of modern software engineering methodologies on a real Arabic project serving local needs.

1.6 Research Methodology and Scientific Methods

This project relied on several integrated research methodologies:

Descriptive Analytical Method:

To study the current reality of the Syrian real estate market, analyze its problems, and review existing solutions globally and regionally.

Engineering Method:

To design and develop the system according to software engineering standards, from requirements analysis to implementation and testing, with complete documentation of each stage.

Experimental Method:

To test the system in different environments, measure its performance and efficiency, and collect user feedback.

1.7 Document Organizational Structure

This thesis consists of six main sections distributed over ten chapters, each addressing a specific aspect of the project, summarized as follows:

- Section One (Chapters 1-2): Provides a general introduction to the project and its development and management methodology.
- Section Two (Chapters 3-4): Reviews previous studies and analyzes system requirements in detail.
- Section Three (Chapter 5): Addresses the architectural design of the system and database.
- Section Four (Chapters 6-8): Documents the implementation environment, code, and test results.
- Section Five (Chapters 9-10): Presents final results and concludes with future development recommendations.
- Section Six: Contains the appendices and references used in the research.

1.8 Basic Definitions and Concepts

- Django: An open-source web framework written in Python, following the MVT pattern.
- MVT (Model-View-Template): An architectural pattern separating data (Model), business logic (View), and presentation (Template).

- ORM (Object Relational Mapping): A technique allowing interaction with databases using programming objects instead of direct SQL.
- SQLite: A lightweight relational database system operating within a single file, suitable for small and medium applications.
- Bootstrap: An open-source CSS framework for quickly building responsive web interfaces.
- Responsive Design: A design that adapts to different screen sizes.
- CRUD: The basic operations on data (Create, Read, Update, Delete).
- HTTP: HyperText Transfer Protocol, the foundation of data communication on the web.
- Session: A server-side mechanism for maintaining user state across multiple requests.
- Migration: A versioned change to the database schema managed by Django's migration framework.
- Template Engine: The component that merges dynamic data with HTML templates to produce final pages.

1.9 Significance of the Syrian Context

While many real estate platforms exist globally, very few address the specific needs of the Syrian market. The Syrian context introduces unique requirements that generic international platforms do not satisfy. The currency is the Syrian Pound, which has its own formatting conventions and order of magnitude that differ substantially from the US Dollar or Euro. The administrative division of the country into fourteen governorates is a fundamental organizing principle for any local real estate search, yet international platforms either ignore Syrian geography entirely or represent it only at the country level.

Furthermore, the primary language of the target users is Arabic, which requires not merely translation but full right-to-left layout support across every component of the interface, including forms, navigation, tables, and the administrative dashboard. A platform that simply translates English labels without restructuring its layout for RTL reading produces a disorienting and unprofessional experience for Arabic-speaking

users. The Aqari project treats Arabic and RTL support as a first-class design constraint rather than an afterthought, which is a significant differentiator from existing solutions.

The economic situation also shapes the technical constraints of the project. Reliable, high-bandwidth internet access cannot be assumed for all users, which motivates a lightweight, server-rendered architecture that minimizes client-side payload and works acceptably on modest connections and older devices. This is a deliberate engineering decision aligned with the realities of the deployment environment, and it informs the choice of a server-side Django architecture over a heavy single-page application framework.

1.10 Expected Beneficiaries

The Aqari platform is expected to benefit several categories of users. Property owners gain a free, professional channel to advertise their properties to a targeted local audience without depending on intermediaries. Property seekers gain a single, organized place to compare options across governorates, filter by precise criteria, and benefit from the accumulated rating feedback of other users. Real estate professionals can use the platform as a complementary tool to broaden their reach. Finally, the academic community benefits from a fully documented, open-source reference implementation that demonstrates how modern software engineering practices can be applied to a locally relevant problem, serving as a learning resource for future students.

1.11 State of the Art in Real Estate Platforms

Modern real estate platforms globally are witnessing a major transformation toward incorporating advanced technologies to enhance user experience and data accuracy. The prominent current trends (State of the Art) in this field can be summarized as follows:

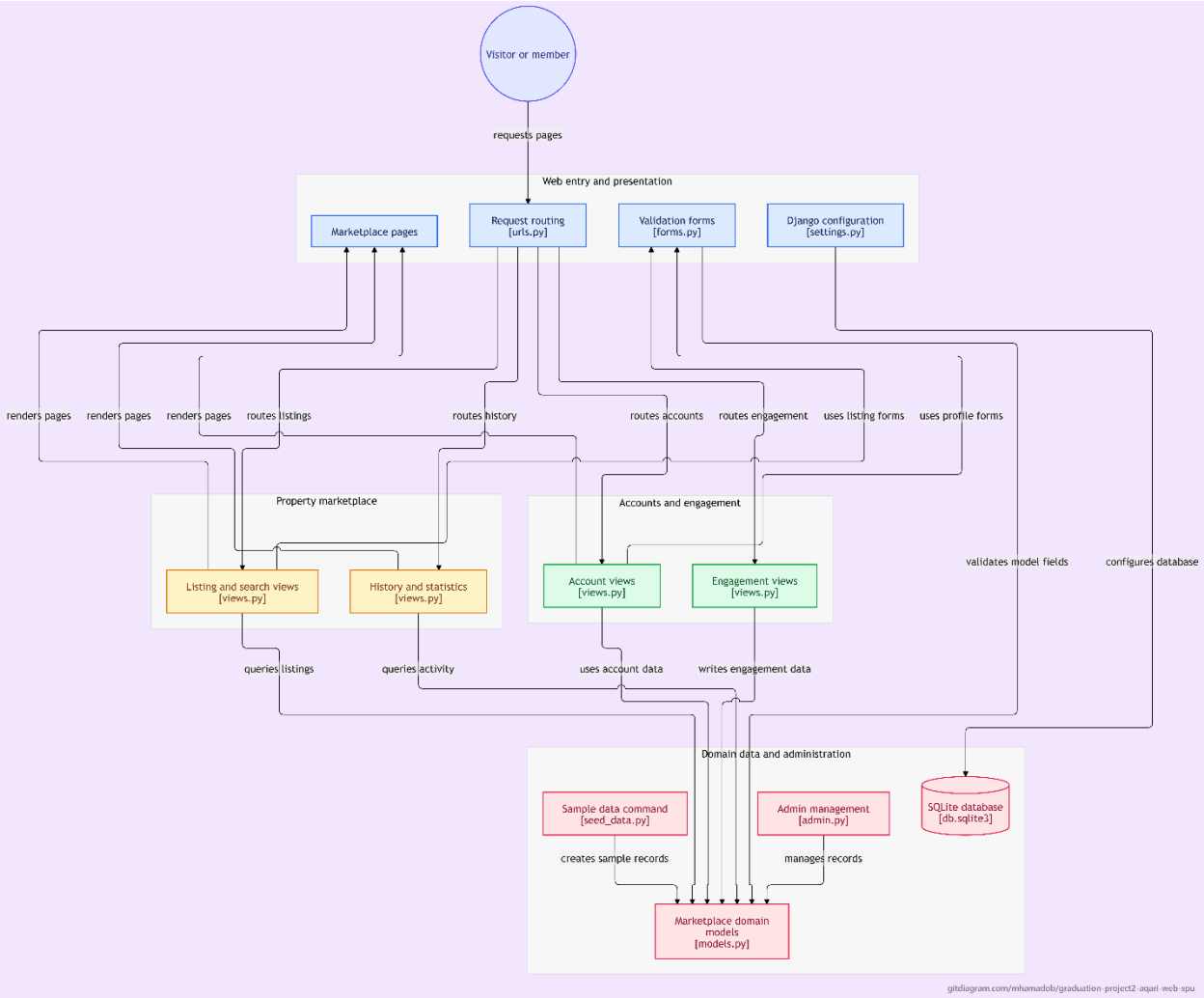
1. **AI-Driven Recommendation Engines:** Utilizing machine learning algorithms to deliver personalized recommendations for each user based on historical browsing behavior, price preferences, and geographical location.
2. **3D Virtual Tours & Augmented Reality (AR):** Integrating technologies such as Matterport to offer interactive 3D virtual tours, allowing users to explore the property remotely without needing a physical visit.
3. **Automated Valuation Models (AVM) & Predictive Analytics:** Relying on Big Data analytics to estimate the fair market value of a property based on economic shifts, neighboring property prices, and historical regional data.

4. **Lightweight Architecture & Server-Side Rendering (SSR):** Leveraging Server-Side Rendering via frameworks such as Django or Next.js to ensure performance optimization on low-bandwidth networks and improve Search Engine Optimization (SEO).

Positioning of the "Aqari" Project Relative to the State of the Art:

The "Aqari" project represents a foundational step in transitioning local real estate listings from unorganized, random posts into a smart platform built on dynamic rendering and real user interaction. While the current scope of the project focuses on addressing basic local infrastructure and connectivity constraints—by delivering a lightweight SSR architecture and a data-driven rating and interaction system—it establishes the core architectural, database, and system foundations (MVT) that enable seamless integration of AI recommendation engines and predictive valuation analytics in future iterations.

1.12 System Architecture & Block Diagram Description



The block diagram illustrates the software architecture of the "Aqari" platform based on Django's **MVT (Model-View-Template)** architectural pattern. It depicts the end-to-end request/response lifecycle from user interaction to data persistent storage and back. The system is structured into four primary layers:

1. User Interaction & Web Entry Layer

- **Visitor or Member:** Represents the end-user initiates requests (`requests pages`).
- **Web Entry and Presentation:** Comprises core Django routing and configuration entry points:
  - **Request Routing (`urls.py`):** Intercepts incoming HTTP requests and routes them to their corresponding view logic.
  - **Form Validation (`forms.py`):** Validates and sanitizes user input and form submissions.
  - **System Settings (`settings.py`):** Holds global system configurations and database connections.
  - **Marketplace Pages:** Represents the user-facing interface templates.

2. Business Logic & Processing Layer (Views)

This layer handles core application logic and is split into two functional domains:

- **Property Marketplace Domain:**
  - **Listing and Search Views (`views.py`):** Manages property queries (`queries listings`), filtering, and multi-criteria search by governorate, property type, price, and status.
  - **History and Statistics Views (`views.py`):** Tracks user activity and statistics (`queries activity`).
- **Accounts and Engagement Domain:**
  - **Account Views (`views.py`):** Handles user account management and profiles (`uses account data`).
  - **Engagement Views (`views.py`):** Manages user interactions such as ratings, favorites, and feedback (`writes engagement data`).

3. Domain Data & Administration Layer

The backend persistence and administrative layer of the system:

- **Marketplace Domain Models (`models.py`):** Defines the database schemas and relationships for properties, users, and interaction data.
- **Data Management & Seeding:**
  - **Sample Data Command (`seed_data.py`):** Generates representative test data for development (`creates sample records`).
  - **Admin Management (`admin.py`):** Configures the administrative dashboard for managing database records (`manages records`).
- **Relational Database (`db.sqlite3`):** An SQLite file-based database storing persistent system data, schema constraints, and model instances.

4. Routing & Presentation Pipeline

Inter-layer connectors (`routes listings`, `routes history`, `routes accounts`) manage data flow between entry points and business logic, while the rendering engine merges model data into templates (`renders pages`) to return dynamic HTML responses to the end user.

users. The Aqari project treats Arabic and RTL support as a first-class design constraint rather than an afterthought, which is a significant differentiator from existing solutions.

The economic situation also shapes the technical constraints of the project. Reliable, high-bandwidth internet access cannot be assumed for all users, which motivates a lightweight, server-rendered architecture that minimizes client-side payload and works acceptably on modest connections and older devices. This is a deliberate engineering decision aligned with the realities of the deployment environment, and it informs the choice of a server-side Django architecture over a heavy single-page application framework.

1.10 Expected Beneficiaries

The Aqari platform is expected to benefit several categories of users. Property owners gain a free, professional channel to advertise their properties to a targeted local audience without depending on intermediaries. Property seekers gain a single, organized place to compare options across governorates, filter by precise criteria, and benefit from the accumulated rating feedback of other users. Real estate professionals can use the platform as a complementary tool to broaden their reach. Finally, the academic community benefits from a fully documented, open-source reference implementation that demonstrates how modern software engineering practices can be applied to a locally relevant problem, serving as a learning resource for future students.

Chapter 2

Project Development & Management Methodology

Objective: To demonstrate that the project was conducted in a professional, systematic manner, not randomly.

2.1 Selected Development Methodology and Justification

After studying the main software engineering methodologies, the Agile methodology within the Scrum framework was chosen to develop the "Aqari" project, for several reasons:

- Flexibility in handling requirements: as the project's requirements were evolving and expanding during the different development stages.
- Iterative development: allowing the project to be completed in stages (Sprints), each lasting one to two weeks.
- Continuous feedback: where each stage can be reviewed with the supervisor and feedback obtained.
- Focus on the working product: producing a runnable version after each sprint.

Comparison between Agile and Waterfall:

Criterion	Waterfall	Agile (Scrum)
Flexibility	Low - requirements fixed early	High - requirements evolve
Delivery	Complete at the end	Incremental after each sprint
Feedback	At project end	After each sprint
Change Management	Difficult and costly	Easy and expected
Suitable for	Clear, stable projects	Evolving projects

2.2 Project Timeline

The project work was divided into several stages (Sprints) distributed over a full academic semester:

Sprint	Main Tasks	Duration
Sprint 1	Requirements study and analysis	2 weeks
Sprint 2	Database and architecture design	2 weeks

Sprint	Main Tasks	Duration
Sprint 3	Models and core backend development	3 weeks
Sprint 4	UI and templates construction	3 weeks
Sprint 5	Rating, favorites & search features	2 weeks
Sprint 6	Admin panel development	2 weeks
Sprint 7	Testing and bug fixing	2 weeks
Sprint 8	Documentation and final report	2 weeks

2.3 Risk Management

Risk	Probability	Impact	Mitigation Plan
Data loss	Medium	High	Daily DB backup, use Git
Delivery delay	Medium	High	Split into sprints, weekly reviews
Tech complexity	Low	Medium	Choose well-documented technologies
Compatibility issues	Low	Medium	Test on multiple browsers/devices
Requirements change	High	Medium	Adopt flexible Agile methodology

2.4 Team and Role Distribution

Since the project is individual, the student performed all the following roles:

- Systems Analyst: requirements analysis and feasibility study.
- Software Designer: architecture and ERD design.
- Backend Developer: writing Python and Django code.
- Frontend Developer: designing and developing interfaces with HTML/CSS/JavaScript.
- Database Administrator (DBA): managing SQLite and queries.
- QA Engineer: testing the system and documenting errors.
- Technical Writer: writing the report and documenting the code.

2.5 Quality Management

To ensure the quality of the final product, several mechanisms were adopted:

- Code writing standards: following PEP 8 for Python code, and HTML/CSS organization.
- Comments and documentation: documenting each function and software unit clearly.
- Comprehensive manual testing: testing each feature before moving to the next stage.
- Self Code Review: re-reading the code after a period to improve it.
- Use of code analysis tools: such as pylint and flake8.

Quality management in this project was understood not as a single phase but as a continuous discipline woven through every sprint. The distinction is important: treating quality as a final inspection step, applied only after the system is otherwise complete, consistently produces inferior results because defects introduced early compound and become expensive to remove late. By contrast, building quality in continuously — writing clean code as the default, testing each feature as it is completed, and reviewing code while its design rationale is still fresh in mind — keeps the system in a consistently releasable state and prevents the accumulation of technical debt.

The adherence to a recognized style guide deserves emphasis as a quality mechanism rather than a cosmetic preference. A consistent code style reduces the cognitive effort required to read and modify code, which directly reduces the rate at which defects are introduced during maintenance. When every part of the codebase follows the same conventions for naming, structure, and formatting, a developer's attention is free to focus on logic rather than on deciphering idiosyncratic style. This is why automated style checking was incorporated into the routine workflow rather than left to ad hoc manual judgment.

Documentation was treated as a first-class quality artifact rather than an afterthought. Code-level documentation explained the reasoning behind non-obvious decisions, so that a future maintainer — including the author after an interval — can understand not merely what the code does but why it does it that way. This project-level thesis itself is the highest tier of that documentation discipline, capturing the complete reasoning

of the project so that its design decisions are auditable and its lessons transferable. Documentation that explains intent, not just mechanics, is what allows a system to be maintained and extended confidently rather than fearfully.

2.6 Project Management Tools Used

- GitHub: for code management and version tracking.
- Trello: for organizing tasks and tracking sprints.
- Notion: for writing notes and internal documents.
- VS Code: as the main development environment.
- Draw.io: for drawing diagrams (UML, ERD).

2.7 Monitoring and Continuous Evaluation Mechanisms

- Weekly meetings with the supervisor to review progress.
- Progress reports at the end of each sprint: accomplishments, challenges, next plans.
- Functional testing after each new feature.
- Self code review every week.

2.8 Software Configuration Management (SCM)

Git was used with GitHub to manage the code, adopting an organized branching strategy:

- main: the main branch containing the stable version.
- dev: the main development branch.
- feature/*: separate branches for each new feature (e.g., feature/rating-system).
- bugfix/*: branches for fixing errors.

Each version was documented with a clear commit message explaining the changes, and Semantic Versioning was adopted for releases. Semantic Versioning follows the MAJOR.MINOR.PATCH format, where the MAJOR number is incremented for incompatible changes, MINOR for backward-compatible feature additions, and PATCH for backward-compatible bug fixes. This disciplined approach to versioning

made it possible to track exactly which features were available in each iteration of the system, and to roll back safely when a regression was detected during testing.

In addition to branch management, a clear commit convention was followed throughout the project. Each commit message began with a type prefix such as feat, fix, docs, refactor, or test, followed by a short imperative description of the change. For example, a commit adding the rating feature would read "feat: add arithmetic mean rating system for properties", while a commit fixing a display bug would read "fix: correct RTL alignment of property cards on mobile". This convention produced a readable and meaningful project history that served as living documentation of the development process.

The repository was also protected with a clear .gitignore configuration to prevent sensitive or unnecessary files from being committed. This included the SQLite database file, the Python virtual environment directory, compiled bytecode caches, IDE configuration folders, and user-uploaded media. Keeping these files out of version control reduced the size of the repository, prevented accidental leakage of local data, and ensured that every developer worked from a clean and reproducible baseline.

2.9 Lessons Learned from the Methodology

Applying the Agile methodology to an individual graduation project provided several valuable insights. First, the iterative nature of Scrum made it possible to demonstrate tangible progress at the end of every sprint, which kept motivation high and made supervisor feedback concrete and actionable rather than abstract. Second, the discipline of writing user stories before implementation forced a user-centered mindset, ensuring that every feature was justified by a real user need rather than added for technical novelty. Third, the sprint retrospective at the end of each cycle surfaced process improvements early, such as the decision to write tests immediately after each feature rather than deferring all testing to the end.

The methodology also exposed the importance of realistic estimation. Early sprints tended to be over-ambitious, with more tasks planned than could be completed in the allotted time. By sprint three, estimation had improved noticeably as historical velocity data accumulated, allowing each subsequent sprint to be planned with greater accuracy. This experience reinforced a core Agile principle: estimation is a skill that

improves with measured feedback, not a one-time prediction made at the start of a project.

2.10 Adaptation of Scrum for a Single Developer

Standard Scrum is designed for teams of several members with distinct roles such as Product Owner, Scrum Master, and Development Team. Adapting it for a single developer required pragmatic modifications. The student assumed all three roles simultaneously: acting as Product Owner when prioritizing the backlog and defining acceptance criteria, as Scrum Master when removing impediments and protecting the sprint scope, and as the Development Team when implementing and testing the work.

The daily stand-up meeting, normally a synchronization ritual among team members, was replaced by a brief daily written log in which the student recorded what was accomplished the previous day, what was planned for the current day, and any blockers encountered. This lightweight practice preserved the reflective benefit of the stand-up without the overhead of a meeting, and produced a continuous written record that later proved invaluable when writing this documentation.

Sprint reviews were conducted with the academic supervisor, who effectively played the role of a stakeholder evaluating the increment. This arrangement preserved the essential Scrum feedback loop and ensured that the direction of the project remained aligned with academic expectations and real-world usefulness throughout its lifecycle.

SECTION TWO

Theoretical Study & Analysis

Chapter 3

Literature Review & Previous Studies

Objective: To prove the project is based on prior knowledge and fills a real gap in its field.

3.1 Previous Studies in Real Estate Systems

Electronic real estate platforms are among the most vital and growing fields in the software industry. This field has produced many studies and research papers; we review the most important below:

Study 1: Smart Real Estate Platform Using AI (2022)

Researchers presented a real estate system using machine learning algorithms to predict property prices. The system relied on a Random Forest model and achieved 87% accuracy in price prediction. Strengths: use of real data and accurate prediction. Weaknesses: no Arabic interface and limited to the US market.

Study 2: Open-Source Real Estate Platform Using Django (2021)

This study proposed building a real estate platform using Django with an advanced search system and user management. Strength: open-source framework that is easy to develop. Weakness: did not address rating or smart analytics features.

Study 3: Real Estate Management System in the Middle East (2023)

This study examined a real estate platform for the Gulf market with features such as most-viewed properties. Strength: Arabic interface. Weakness: does not serve the Syrian market with its local characteristics (governorates, currency).

Study 4: Rating System in E-commerce Platforms (2020)

The study presented a model for a star rating system in e-commerce applications, with arithmetic mean calculation. This technique was adapted and applied in the "Aqari" project.

Study 5: Modern Admin Dashboard Design - Material Design (2021)

Google presented Material Design principles for designing modern admin dashboards, which the project benefited from in designing the "Aqari" admin dashboard.

Study 6: Responsive Web Design for Emerging Markets (2022)

This study examined how responsive design and progressive enhancement improve accessibility for users in regions with constrained bandwidth and older devices. It

concluded that server-rendered, lightweight pages substantially outperform heavy client-side applications in such contexts. This finding directly informed the architectural choice of a server-rendered Django approach in the Aqari project, and validated the decision to minimize client-side payload.

Study 7: Trust and Reputation Systems in Online Marketplaces (2020)

This research analyzed how rating and reputation mechanisms increase user trust and transaction confidence in online marketplaces. It demonstrated empirically that even a simple aggregate rating, when presented transparently, measurably increases user engagement and willingness to transact. This study provides the theoretical justification for the prominence given to the arithmetic mean rating feature in Aqari.

Study 8: Arabic Natural Language Interfaces (2023)

This study addressed the specific usability considerations of Arabic-language software, emphasizing that correct right-to-left layout, appropriate typography, and culturally appropriate iconography are essential for user acceptance in Arabic-speaking markets. Its conclusions reinforced the decision to treat Arabic RTL support as a first-class design constraint rather than a cosmetic translation layer.

Synthesis of the Literature

Taken together, the reviewed studies establish three converging conclusions that shaped this project. First, the technical feasibility of building a robust real estate platform with open-source web frameworks is well established, removing technical risk from the framework choice. Second, intelligent aggregate features such as ratings and popularity ranking provide measurable user value and are therefore worth the implementation effort. Third, and most importantly, no existing solution in the literature or the market combines these capabilities with genuine localization for the Syrian market — complete governorate coverage, local currency, and professional Arabic RTL support. This precise combination is the gap that the Aqari project fills, and the literature review confirms that the gap is real rather than assumed.

3.2 Modern Technologies and Frameworks

- Django (Python): A powerful framework for building web applications, featuring high security and rapid development.

- Laravel (PHP): A popular framework for web development, using the MVC pattern.
- Ruby on Rails: A framework focusing on productivity and less code.
- Node.js + Express: For building real-time applications.
- React / Vue / Angular: Libraries for building interactive user interfaces.
- Bootstrap / Tailwind: CSS frameworks to speed up interface building.

3.3 Comparative Analysis of Existing Systems

Feature	Zillow	OpenSooq	Bayut	Aqari
Arabic Interface	No	Yes	Yes	Yes
Syrian Market	No	Partial	No	Yes
Rating System	No	No	Yes	Yes
Most-Viewed (real data)	Yes	Yes	Yes	Yes
Open Source	No	No	No	Yes
Local Admin Panel	No	No	No	Yes
14 Syrian Governorates	No	No	No	Yes
Syrian Pound Currency	No	No	No	Yes

3.4 Research Gap and Proposed Innovation

Based on the previous analysis, it is clear that the Syrian real estate market lacks a comprehensive specialized platform. The main gaps:

13. Absence of a unified platform serving all 14 Syrian governorates.
14. No solution in the local currency (Syrian Pound).
15. Absence of smart tools such as rating and displaying real view data.

16. Existing systems lack a professional Arabic interface with a complete Arabic admin panel.

The "Aqari" project comes to fill this gap by providing an integrated, local, open-source platform.

3.5 Theoretical Framework and Reference Studies

Every engineering project rests on a foundation of established theory, and making that foundation explicit is essential for academic rigor. The Aqari platform draws on several well-established theoretical pillars, each of which is examined below in terms of both its abstract principle and its concrete application within this project.

MVT Pattern (Model-View-Template):

An architectural pattern separating data (Model) from business logic (View) from presentation (Template), the pattern used by Django.

The deeper theoretical principle underlying MVT is the separation of concerns, a foundational idea in software engineering which holds that a system should be decomposed into parts such that each part addresses a distinct concern with minimal overlap. The benefit is not aesthetic but practical: when concerns are properly separated, a change to one concern — for example, redesigning the visual appearance — can be made without risk of inadvertently breaking an unrelated concern such as the business rules. In the Aqari implementation this principle is honored strictly: the templates contain no business logic, the views contain no raw database access, and the models contain no presentation code. This discipline is what makes the system maintainable over time rather than progressively more fragile with each change.

REST Architecture:

An architectural style for designing web services, relying on HTTP Methods (GET, POST, PUT, DELETE).

The relevant theoretical insight from REST adopted in this project is the principle of meaningful, resource-oriented addressing and the appropriate use of HTTP semantics. Each significant resource in the system — a property, a user's favorites, a

search — has a clean, predictable URL, and the HTTP method communicates intent: a safe, side-effect-free retrieval uses GET, while a state-changing operation uses POST. Adhering to these semantics produces a system whose behavior is predictable to both developers and the web infrastructure itself, including browsers and caches, which is a tangible benefit derived directly from the underlying theory.

Relational Database Theory:

The data layer rests on relational theory, in which data is organized into relations (tables) connected by keys, and integrity is enforced through constraints. The project applies the theory of normalization to eliminate redundant data: rather than repeating user information within every property record, the user is stored once and referenced by key. This eliminates the update anomalies that plague denormalized designs, where the same fact stored in multiple places can become inconsistent. The foreign-key constraints and the uniqueness constraint on ratings are direct applications of relational integrity theory that guarantee the database can never enter certain classes of invalid states.

Arithmetic Mean for Ratings:

Average Rating = Sum(Ratings) / Count(Ratings) - calculated from values 1 to 5.

Although the arithmetic mean is elementary mathematically, its selection over alternatives was a considered theoretical decision. Alternatives such as a Bayesian average or a time-weighted score offer advantages in very large systems with sparse or adversarial data, but they introduce complexity and reduce transparency. For a platform at this stage, the unweighted arithmetic mean has the decisive virtue of being immediately understandable to users: a four-star average means exactly what an ordinary person expects it to mean. This deliberate choice of the simplest model that adequately solves the problem reflects the engineering principle that complexity must be justified by necessity, not adopted for its own sake.

3.6 Modern and Future Trends

- Artificial Intelligence (AI): predicting and recommending property prices.
- Augmented Reality (AR): virtual 3D tours of properties.
- Big Data Analytics: analyzing big data for decision-making.

- Blockchain: for documenting sales contracts and ensuring security.
- IoT (Internet of Things): integrating data from sensors.

Chapter 4

Requirements Modeling & Analysis

Objective: To document what the system must do and how it will be verified.

4.1 Feasibility Study

Technical Feasibility:

The system is built with well-known open-source technologies (Django, Python, SQLite, Bootstrap), all available, free, and well-documented. There are no technical constraints preventing the implementation of the system with the proposed features.

Financial Feasibility:

The approximate costs of the system:

Item	Annual Cost (SYP)
Website hosting (VPS server)	1,500,000
Domain name	300,000
SSL certificate	200,000
Backup and maintenance	500,000
Total annual	2,500,000

Operational Feasibility:

The system is easy to use for ordinary users thanks to its Arabic interface, and does not require intensive training.

4.2 Requirements Gathering

Requirements were gathered using several methods:

- Personal interviews: with 5 people experienced in the Syrian real estate market.
- Questionnaires: an electronic survey distributed to 30 potential users.
- Observations: studying user behavior on existing real estate platforms.
- Desk research: reviewing previous studies and best practices in the field.

4.3 Stakeholder Identification

Stakeholder	Needs	Expectations
End User (Seeker)	Browse, search, favorites	Speed, ease, data accuracy
Property Owner	Easily post listing, reach clients	Simple interface, professional display
Admin	Manage listings, remove abuse	Powerful tools, reports, alerts
Future Developer	Clean code, documentation	Clear structure, full docs
University	Achieve graduation objectives	Complete documented project

4.4 Functional Requirements

Code	Requirement	Priority
FR1	User can register a new account	Must
FR2	User can log in and out	Must
FR3	User can edit personal data	Should
FR4	User can delete account	Should
FR5	User can change password	Should
FR6	User can add a new property	Must
FR7	Owner can edit their property	Must
FR8	Owner can delete their property	Must
FR9	System displays full property details	Must
FR10	System displays all properties on home page	Must
FR11	User can rate property from 1 to 5 stars	Must
FR12	System auto-calculates average rating	Must
FR13	System provides keyword search	Must
FR14	System filters by governorate/type/price	Must
FR15	User can add property to favorites	Must

Code	Requirement	Priority
FR16	System displays user's favorites list	Must
FR17	User can remove from favorites	Should
FR18	User can report a violating property	Should
FR19	System displays real view count	Must
FR20	System auto-ranks most-viewed properties	Must
FR21	System provides owner profile page	Should
FR22	System supports property image upload	Must
FR23	Admin panel provides full property management	Must
FR24	Admin panel provides reports and statistics	Should
FR25	System supports 14 Syrian governorates	Must

4.5 Non-Functional Requirements

Aspect	Requirement
Performance	Response time < 2 seconds per page
Performance	Support at least 100 concurrent users
Security	Password encryption using PBKDF2
Security	Protection from CSRF, XSS, SQL Injection
Security	Clear roles (user, owner, admin)
Usability	Full Arabic interface with RTL support
Usability	Responsive design for all devices
Usability	Add a property in less than 3 minutes
Availability	99% system availability
Maintainability	Clean, documented code following PEP 8
Compatibility	Works on Chrome, Firefox, Safari, Edge

4.6 Requirements Prioritization (MoSCoW)

Category	Count	Examples
Must Have	16	Login, add property, search, rating
Should Have	7	Edit account, change password, reporting
Could Have	2	Owner page, advanced statistics
Won't Have	—	Mobile app, e-payment, AI prediction

4.7 UML Diagrams for Requirements

4.7.1 Main Use Case Diagram

The following diagram illustrates the main actors (Guest, Registered User, Admin) and their interactions with the "Aqari" system. The use cases in red are restricted to registered users only, while green ones are admin-exclusive.

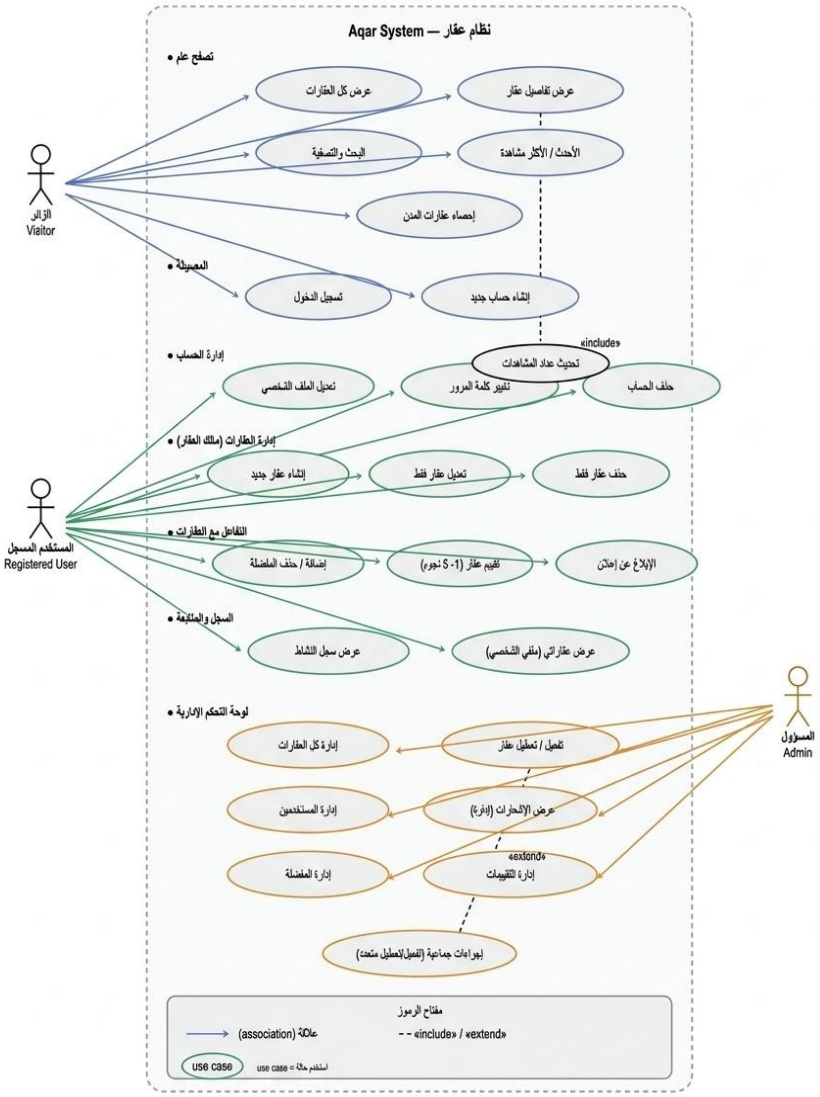


Figure 4-1: Use Case Diagram - Aqari System

4.7.2 Use Case Descriptions

Name	Actor	Main Flow
Add Property	Owner	Login → Open add form → Fill data → Upload image → Save
Rate Property	User	Open property → Select stars → Click submit rating
Search	Any visitor	Enter keyword + filters → Click search
Add to Favorites	User	Open property → Click heart icon
Report Property	User	Open property → Click report → Write reason → Submit

Name	Actor	Main Flow
Manage Users	Admin	Open admin panel → Users section → Edit/Delete

4.7.3 Activity Diagram

The following activity diagram illustrates the workflow of the "Add Property" process, including authentication and data validation decision points:

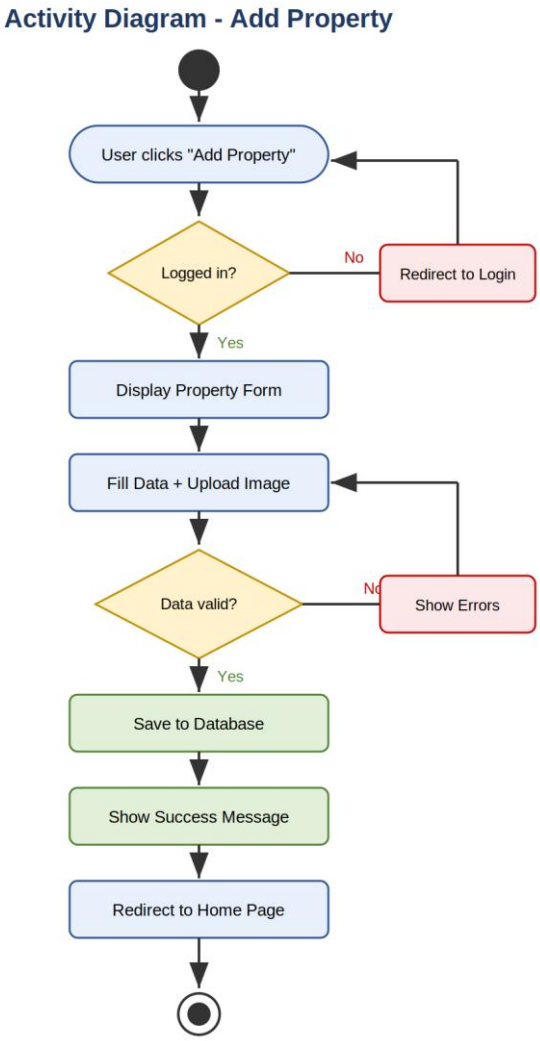


Figure 4-2: Activity Diagram - Add Property Process

4.7.4 Sequence Diagram

The following sequence diagram shows the interaction sequence when a user rates a property, illustrating the message flow between the User, Template, View, Model, and Database, including the average rating recalculation:

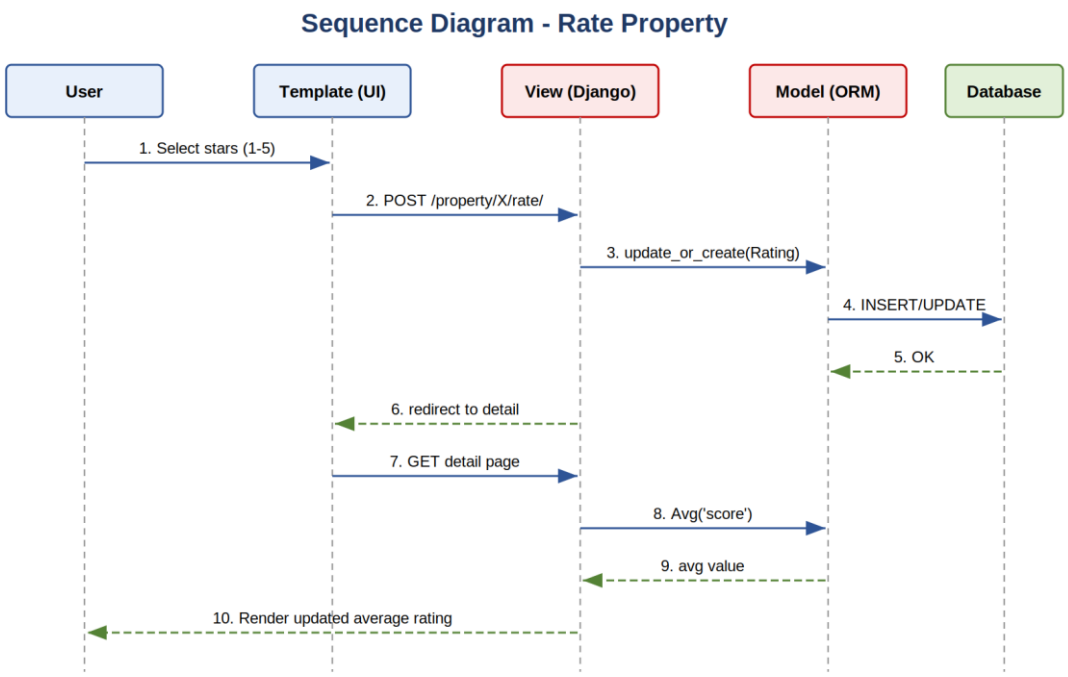


Figure 4-3: Sequence Diagram - Rate Property

4.7.5 Diagram Analysis and Rationale

The use case diagram deliberately distinguishes three actor types by access level rather than by job title, because access level is what actually drives the system's authorization logic. The Guest actor represents any unauthenticated visitor and is intentionally limited to read-only operations: browsing and searching. This open read access is a conscious product decision that maximizes discoverability, since requiring registration merely to view listings would deter the casual seekers who form the majority of the audience. The Registered User actor inherits all Guest capabilities and adds the personalized, state-changing operations: adding properties, rating, managing favorites, and reporting. The Admin actor represents the privileged operator with full system oversight.

The activity diagram for the add-property workflow was chosen for detailed modeling because it contains the richest decision logic in the system. It includes two distinct decision points: an authentication gate that redirects unauthenticated users to the login page, and a validation gate that returns the user to the form with error messages when submitted data is invalid. Modeling these decision points explicitly clarifies the control flow and exposes the loop-back paths that are easy to overlook in prose but critical to a correct and user-friendly implementation. The diagram also makes clear that a

successful submission ends with both a persistence action and a user-facing confirmation, which together constitute a complete and satisfying interaction.

The sequence diagram for the rating workflow was selected because it best illustrates the collaboration between the MVT layers over time. It shows ten ordered messages flowing across five participants, beginning with the user's star selection and ending with the re-rendered page displaying the updated average. Critically, it visualizes the point at which the average is recomputed: not at write time, but at the next read of the detail page, using a database aggregation. This design keeps the write path simple and the displayed average always consistent with the stored ratings, and the sequence diagram makes this otherwise subtle design decision visible and reviewable.

Together, these three diagrams cover the three complementary views that UML is designed to express: the use case diagram captures the static set of system capabilities and who may invoke them, the activity diagram captures the dynamic control flow within a single complex operation, and the sequence diagram captures the dynamic collaboration between components over time. Selecting one representative diagram of each type, rather than exhaustively diagramming every trivial operation, follows the guidance that models should illuminate the parts of a system that genuinely benefit from visualization rather than mechanically document the obvious.

4.8 Modeling According to the Used Methodology

Requirements modeling must align with the development methodology actually employed. Because this project adopted the Agile (Scrum) methodology as its primary approach, the requirements were modeled using Agile artifacts. However, for completeness and to demonstrate command of the full spectrum of requirements engineering, this section also presents the traditional Software Requirements Specification (SRS) format and the iterative Feature Lists format, so that the requirements are documented from three complementary perspectives.

4.8.1 For Agile Methodologies: User Stories, Story Mapping, Product Backlog

User Stories: In the Agile approach, requirements are expressed as short, user-centered statements following the canonical template "As a [role], I want [capability] so that [benefit]". This format keeps the focus on user value rather than technical

implementation. The complete set of user stories for the Aqari system is presented below, organized by epic.

Epic 1 — Account Management:

- US-01: As a visitor, I want to register a new account so that I can access personalized features.
- US-02: As a registered user, I want to log in and out so that I can securely access my account.
- US-03: As a registered user, I want to edit my personal data so that my profile stays accurate.
- US-04: As a registered user, I want to change my password so that I can keep my account secure.
- US-05: As a registered user, I want to delete my account so that I can remove my data when I choose.

Epic 2 — Property Management:

- US-06: As an owner, I want to add a property with its details and photo so that I can reach potential buyers or tenants.
- US-07: As an owner, I want to edit my property so that I can keep the listing information current.
- US-08: As an owner, I want to delete my property so that I can remove it once it is no longer available.
- US-09: As an owner, I want a profile page showing all my listings so that buyers can see everything I offer.

Epic 3 — Browsing and Search:

- US-10: As a seeker, I want to browse all properties on the home page so that I get an overview of what is available.
- US-11: As a seeker, I want to search by keyword so that I can quickly find relevant listings.
- US-12: As a seeker, I want to filter by governorate, type, status, and price so that I can narrow results to my needs.

- US-13: As a seeker, I want to view full property details so that I can evaluate it before contacting the owner.
- US-14: As a seeker, I want to see the most-viewed properties so that I know what is popular.

Epic 4 — Interaction and Trust:

- US-15: As a user, I want to rate a property from one to five stars so that I can share my experience.
- US-16: As a user, I want to see the average rating of each property so that I can judge its quality at a glance.
- US-17: As a user, I want to add a property to my favorites so that I can revisit it later.
- US-18: As a user, I want to view and manage my favorites list so that I can organize properties of interest.
- US-19: As a user, I want to report a violating listing so that the platform stays trustworthy.

Epic 5 — Administration:

- US-20: As an admin, I want a modern dashboard so that I can manage all listings efficiently.
- US-21: As an admin, I want to activate or deactivate listings so that I can moderate content non-destructively.
- US-22: As an admin, I want to review user reports so that I can act on problematic listings.

Story Mapping: A story map arranges user stories two-dimensionally to visualize the product. The horizontal axis represents the user's journey through the main activities in the order they are typically performed, while the vertical axis represents priority, with the highest-priority stories forming the top row that constitutes the minimum viable product (MVT). The story map for Aqari is presented below.

Discover	Register	List Property	Search	Evaluate	Interact	Administer
Browse home (US-10)	Register (US-01)	Add property (US-06)	Keyword search (US-11)	View details (US-13)	Rate property (US-15)	Admin dashboard (US-20)
Most-viewed (US-14)	Login/Logout (US-02)	Edit property (US-07)	Filter results (US-12)	See avg rating (US-16)	Add favorite (US-17)	Activate/Deactivate (US-21)
—	Edit profile (US-03)	Delete property (US-08)	—	—	Manage favorites (US-18)	Review reports (US-22)
—	Change password (US-04)	Owner page (US-09)	—	—	Report listing (US-19)	—

The first row of the map constitutes the MVP: the smallest coherent set of stories that delivers end-to-end value, allowing a user to discover, register, list, search, evaluate, interact, and be administered. Subsequent rows represent successive enhancement releases that deepen each activity without being required for the core value proposition.

Product Backlog: The product backlog is the single, ordered list of all work to be done, with each item assigned a priority using the MoSCoW scheme and an estimated effort in story points. The backlog was continuously re-ordered across sprints as understanding improved. Its final state is shown below.

ID	Backlog Item	Priority	Points	Sprint
US-01	User registration	Must	5	S1
US-02	Login / Logout	Must	3	S1
US-06	Add property + image	Must	8	S3
US-10	Browse all properties	Must	5	S3
US-13	View property details	Must	5	S3

ID	Backlog Item	Priority	Points	Sprint
US-11	Keyword search	Must	5	S5
US-12	Multi-criteria filter	Must	8	S5
US-15	Rate property (1-5)	Must	5	S5
US-16	Average rating display	Must	5	S5
US-14	Most-viewed ranking	Must	5	S5
US-17	Add to favorites	Must	3	S5
US-18	Manage favorites list	Must	3	S5
US-07	Edit property	Must	5	S3
US-08	Delete property	Must	3	S3
US-20	Admin dashboard	Must	8	S6
US-21	Activate/Deactivate listings	Must	3	S6
US-03	Edit profile	Should	3	S4
US-04	Change password	Should	2	S4
US-05	Delete account	Should	2	S4
US-19	Report a listing	Should	3	S6
US-22	Review reports	Should	5	S6
US-09	Owner profile page	Could	5	S4

4.8.2 For Traditional Methodologies: Software Requirements Specification (SRS)

Although this project followed Agile, a Software Requirements Specification is presented here in the traditional IEEE 830 structure to document the requirements from the formal perspective and to demonstrate mastery of the classical approach. The SRS is a contract-style specification that describes precisely what the system must do, independent of how it will be implemented.

SRS 1. Introduction

1.1 Purpose: This document specifies the software requirements for the Aqari real estate platform. It is intended for the developer, the academic supervisor, and any future maintainer, and serves as the authoritative reference against which the delivered system is validated.

1.2 Scope: The Aqari software is a web-based platform that enables registered users to publish, browse, search, rate, and favorite real estate listings across the fourteen Syrian governorates, and enables administrators to moderate content. The software does not include native mobile applications, electronic payment, or live messaging in the current version.

1.3 Definitions: Listing — a property advertisement; Owner — the registered user who created a listing; Guest — an unauthenticated visitor; Admin — a privileged operator; Rating — an integer score from 1 to 5; SYP — Syrian Pound.

1.4 References: IEEE Std 830-1998 Recommended Practice for Software Requirements Specifications; Django Documentation; the project's user stories and use case models.

1.5 Overview: Section 2 describes the overall product perspective and constraints; Section 3 enumerates the specific functional and non-functional requirements; Section 4 defines external interface requirements.

SRS 2. Overall Description

2.1 Product Perspective: Aqari is a self-contained web application built on the Django framework using a server-rendered MVT architecture with an SQLite database. It does not depend on external paid services and operates as a standalone system accessible through a standard web browser.

2.2 Product Functions: The principal functions are account management, property listing management, multi-criteria search and browsing, an arithmetic-mean rating system, a real-data popularity ranking, favorites management, a reporting mechanism, and an administrative moderation dashboard.

2.3 User Characteristics: Users are assumed to possess basic computer or smartphone literacy and an internet connection. No specialized technical knowledge is required. The primary language of all users is Arabic, necessitating full right-to-left interface support.

2.4 Constraints: The system must be implemented with free open-source technologies; it must operate acceptably on modest internet connections and older devices; and it must be deliverable within one academic year by a single developer.

2.5 Assumptions and Dependencies: It is assumed that users act honestly when publishing listings, that the hosting environment can support a moderate number of concurrent users, and that the Python and Django runtime is available in the deployment environment.

SRS 3. Specific Requirements — Functional

Req ID	Requirement Statement	Source US	Priority
SRS-F-01	The system shall allow a visitor to create an account with a unique username and password.	US-01	High
SRS-F-02	The system shall authenticate users and maintain a session until logout.	US-02	High
SRS-F-03	The system shall allow a user to update their profile information.	US-03	Medium
SRS-F-04	The system shall allow a user to change their password after verifying the current one.	US-04	Medium
SRS-F-05	The system shall allow a user to permanently delete their account.	US-05	Medium
SRS-F-06	The system shall allow an authenticated user to create a listing with title, description, price, governorate, address, area, rooms, type, status, and an optional image.	US-06	High
SRS-F-07	The system shall allow only the owner of a listing to edit it.	US-07	High
SRS-F-08	The system shall allow only the owner of a listing to delete it, cascading deletion to its ratings, favorites, and reports.	US-08	High

Req ID	Requirement Statement	Source US	Priority
SRS-F-09	The system shall display all active listings on the home page.	US-10	High
SRS-F-10	The system shall display the full details of a selected listing and increment its view counter.	US-13	High
SRS-F-11	The system shall provide keyword search over listing title and description.	US-11	High
SRS-F-12	The system shall filter listings by governorate, type, status, and price range, combinable simultaneously.	US-12	High
SRS-F-13	The system shall allow an authenticated user to submit one rating from 1 to 5 per listing, updating any prior rating.	US-15	High
SRS-F-14	The system shall compute and display the arithmetic mean of a listing's ratings and the rating count.	US-16	High
SRS-F-15	The system shall rank listings by genuine accumulated view count on the most-viewed page.	US-14	High
SRS-F-16	The system shall allow an authenticated user to add and remove listings from a personal favorites list.	US-17, US-18	High
SRS-F-17	The system shall allow an authenticated user to submit a report with a reason against a listing.	US-19	Medium
SRS-F-18	The system shall provide an administrative dashboard to manage listings, users, ratings, and reports.	US-20	High

Req ID	Requirement Statement	Source US	Priority
SRS-F-19	The system shall allow an administrator to activate or deactivate any listing without deleting it.	US-21	High
SRS-F-20	The system shall provide a public profile page per owner listing all of that owner's active properties.	US-09	Low

SRS 3. Specific Requirements — Non-Functional

Req ID	Non-Functional Requirement
SRS-NF-01	Performance: any page shall render in under 2 seconds under normal load.
SRS-NF-02	Performance: the system shall support at least 100 concurrent users.
SRS-NF-03	Security: passwords shall be stored only as salted cryptographic hashes (PBKDF2).
SRS-NF-04	Security: all state-changing forms shall be protected against CSRF.
SRS-NF-05	Security: all database access shall be parameterized to prevent SQL injection.
SRS-NF-06	Security: authorization shall be enforced server-side for every protected operation.
SRS-NF-07	Usability: the entire interface shall be in Arabic with full RTL layout.
SRS-NF-08	Usability: the layout shall be responsive across desktop, tablet, and mobile.
SRS-NF-09	Usability: a user shall be able to add a property in under 3 minutes.
SRS-NF-10	Reliability: the system shall target 99% availability.
SRS-NF-11	Maintainability: code shall follow PEP 8 and be documented.
SRS-NF-12	Compatibility: the system shall function on Chrome, Firefox, Safari, and Edge.

SRS 4. External Interface Requirements

4.1 User Interfaces: The system shall present a web interface composed of a home page, search page, property detail page, add/edit property forms, favorites page,

owner profile page, authentication pages, and an administrative dashboard, all in Arabic with RTL layout.

4.2 Software Interfaces: The system shall interface with an SQLite database through the Django ORM. No external third-party service interfaces are required in the current version.

4.3 Communications Interfaces: The system shall communicate with clients over the HTTP protocol using standard request and response semantics, with state-changing operations using the POST method and retrieval operations using the GET method.

4.8.3 For Iterative Methodologies: Feature Lists

Iterative and incremental methodologies organize requirements as prioritized feature lists grouped by functional area, where each feature is a deliverable unit of capability scheduled into an iteration. The complete feature list for Aqari is presented below.

#	Feature	Functional Area	Iteration
F-01	Account registration and authentication	Identity	Iteration 1
F-02	Profile and password management	Identity	Iteration 4
F-03	Create property listing with image upload	Listings	Iteration 2
F-04	Edit and delete own listing	Listings	Iteration 2
F-05	Owner public profile page	Listings	Iteration 4
F-06	Home page with all active listings	Discovery	Iteration 2
F-07	Keyword search	Discovery	Iteration 3
F-08	Multi-criteria filtering	Discovery	Iteration 3
F-09	Most-viewed ranking (real data)	Discovery	Iteration 3
F-10	Property detail view with view counter	Discovery	Iteration 2
F-11	Star rating submission (1-5)	Trust	Iteration 3
F-12	Arithmetic mean rating display	Trust	Iteration 3
F-13	Favorites add / remove / list	Engagement	Iteration 3

#	Feature	Functional Area	Iteration
F-14	Report a listing	Trust	Iteration 4
F-15	Administrative dashboard	Administration	Iteration 4
F-16	Listing activation / deactivation	Administration	Iteration 4
F-17	Report review by admin	Administration	Iteration 4
F-18	Responsive RTL Arabic interface	Cross-cutting	All iterations

Presenting the same requirements through three distinct lenses — Agile artifacts, a traditional SRS, and an iterative feature list — demonstrates that the underlying requirements are stable and well-understood regardless of the documentation formalism applied. The Agile artifacts emphasize user value and adaptive prioritization, the SRS emphasizes contractual precision and completeness, and the feature list emphasizes incremental deliverability. All three are mutually consistent and traceable to the same set of elicited needs, which is the strongest possible evidence that the requirements engineering for this project was conducted rigorously.

4.9 Requirements Analysis and Documentation

4.9.1 Requirements Traceability Matrix:

Requirement	Design	Implementation	Test
FR1 (Register)	Signup Page	signup_view()	TC-AUTH-01
FR6 (Add Property)	Add Property Page	add_property()	TC-PROP-01
FR11 (Rating)	Star Rating UI	rate_property()	TC-RATE-01
FR12 (Avg Rating)	Detail Page	Avg('rating__score')	TC-RATE-02
FR15 (Favorites)	Favorite Button	add_favorite()	TC-FAV-01
FR20 (Most-viewed)	Popular Page	popular_view()	TC-POP-01
FR23 (Admin)	Admin Panel	admin.py	TC-ADMIN-01

4.10 Detailed Requirements Discussion

The functional requirements presented in Table 4-2 form the backbone of the system, but each carries implications worth examining in detail. The authentication requirements (FR1 through FR5) establish the trust boundary of the platform. Without a reliable account system, none of the personalized features such as favorites, ratings, or ownership of listings would be possible. Django's built-in authentication framework was chosen specifically because it provides a battle-tested, secure foundation for these requirements, including password hashing, session management, and protection against common authentication attacks, rather than implementing these sensitive mechanisms from scratch.

The property management requirements (FR6 through FR10) define the core value proposition of the platform. The decision to allow any registered user to add a property, rather than restricting this to a special class of verified agents, was deliberate: it lowers the barrier to entry and reflects the reality of the Syrian market where individual owners frequently advertise directly. The ownership enforcement implied by FR7 and FR8 — that only the owner of a property may edit or delete it — is a critical security and integrity requirement that prevents users from tampering with listings they do not own.

The intelligent features (FR11, FR12, FR19, FR20) are what elevate Aqari above a simple listing board. The arithmetic mean rating system transforms isolated individual opinions into an aggregate signal of quality that helps seekers make informed decisions. The real view counter, by tracking genuine user interest rather than relying on arbitrary featured placement, produces an honest and self-correcting popularity ranking. Together these features introduce a data-driven dimension to the platform that aligns with the analytical orientation of the project.

The non-functional requirements deserve equal attention because they determine whether the system is pleasant and safe to use, not merely functional. The performance requirement of sub-two-second response times directly affects user retention; research consistently shows that users abandon slow pages. The security requirements protect both the platform and its users from a class of well-understood web attacks. The usability requirements, particularly full Arabic RTL support, are non-negotiable for the target audience and shaped countless small design decisions throughout the implementation.

4.11 Requirements Validation Approach

Gathering requirements is only valuable if those requirements are subsequently validated against reality. For this project, validation occurred at multiple points. During requirements gathering, the questionnaire responses were cross-checked against the interview findings to identify consistent themes and discard outlier opinions that did not represent the broader user base. During design, each requirement was traced to a specific design element through the traceability matrix, ensuring no requirement was orphaned without a corresponding design decision. During implementation, acceptance criteria derived from the user stories served as concrete checkpoints. During testing, the test cases were mapped directly back to requirements, so that a passing test suite constituted objective evidence that the requirements had been met.

This end-to-end traceability — from elicited need, through user story, design element, implementation, and finally test case — is a hallmark of disciplined requirements engineering. It guards against two common failure modes: building features nobody asked for (scope creep) and forgetting to build features that were requested (scope gaps). The traceability matrix presented earlier is the concrete artifact that makes this discipline auditable.

SECTION THREE

Design & Architecture

Chapter 5

System Architectural Design

Objective: To explain how the system will implement what was analyzed in the previous chapter.

5.1 High-Level Design

5.1.1 General System Architecture

The "Aqari" system consists of three main layers, as illustrated in the architecture diagram below:

- Presentation Layer: HTML, CSS, Bootstrap, JavaScript - displays the interface to the user.
- Business Logic Layer: Django Views - processes user requests and applies business rules.
- Data Layer: Django ORM + SQLite - handles data storage and retrieval.

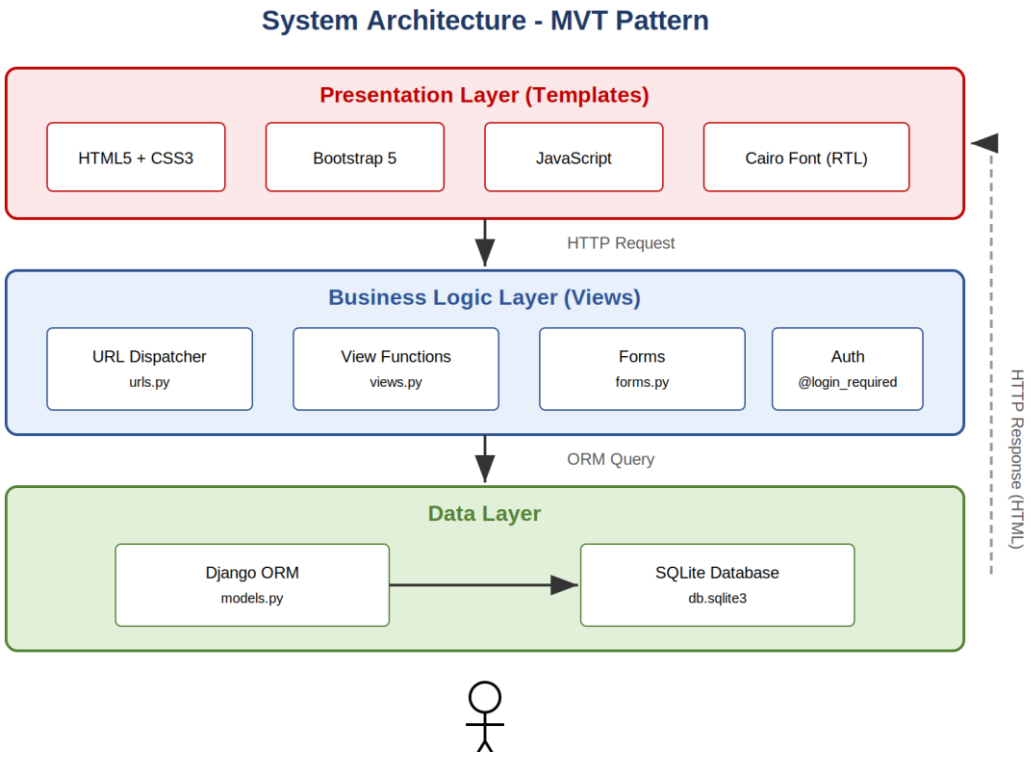


Figure 5-1: System Architecture - MVT Pattern

5.1.2 Architectural Design Pattern Used

The system relied on the MVT (Model-View-Template) pattern used by Django:

- Model: defining the data structure (Property, Favorite, Rating, Report) in models.py.

- View: processing requests and executing business logic in views.py.
- Template: HTML templates for displaying data to the user in the templates/ folder.

5.1.3 System Architecture Description

The interaction begins from the browser where the user sends an HTTP request, received by the Django server which routes it to the URL Dispatcher that determines the appropriate View. The View executes the business logic and queries the database via the ORM, then prepares the context and sends it to the appropriate Template which builds the final page and returns it to the browser.

5.2 Database Design

5.2.1 Entity Relationship Diagram (ERD)

The system consists of 5 main entities. The following diagram shows the entities and the relationships between them:

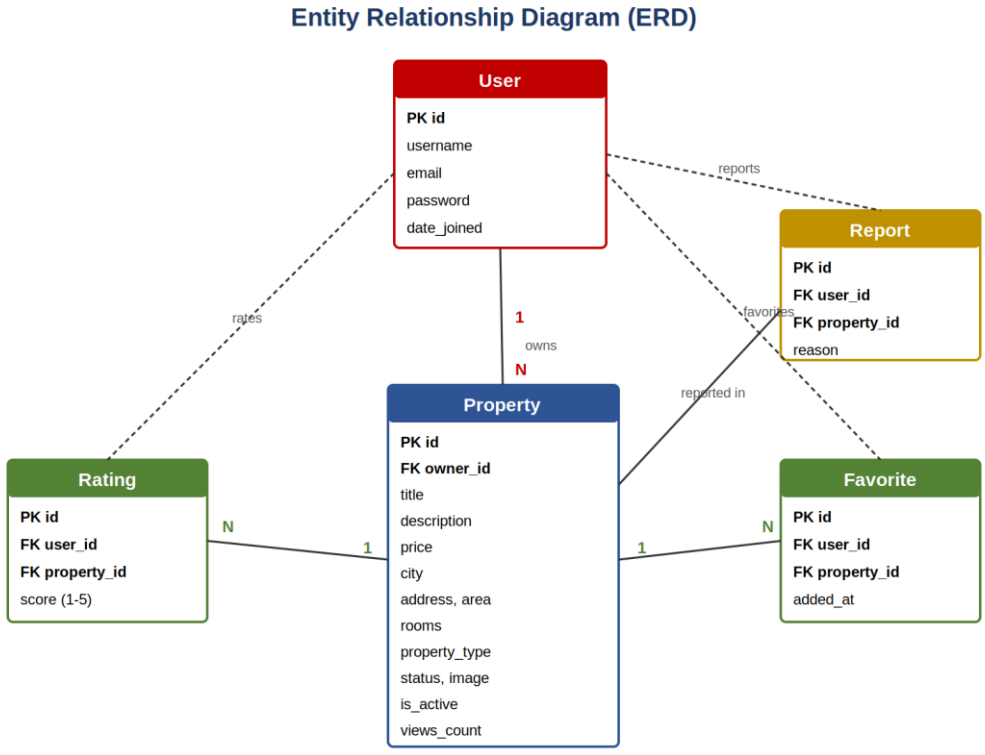


Figure 5-2: Entity Relationship Diagram (ERD)

Relationships between entities:

- User 1 → ∞ Property (one user owns multiple properties).
- User ∞ ↔ ∞ Property via Favorite (Many-to-Many).
- User ∞ ↔ ∞ Property via Rating (Many-to-Many with score value).
- User ∞ ↔ ∞ Property via Report (for recording reports).

5.2.2 Relational Schema

Table: properties_property

Field	Type	Constraint	Description
id	AutoField	PK	Unique property ID
owner_id	ForeignKey	FK → User	Listing owner
title	CharField(200)	NOT NULL	Listing title
description	TextField	NOT NULL	Property description
price	DecimalField	NOT NULL	Price in SYP
city	CharField(50)	NOT NULL	Governorate
address	CharField(255)	NOT NULL	Detailed address
area	FloatField	NOT NULL	Area in m²
rooms	IntegerField	≥0	Number of rooms
property_type	CharField(20)	Choices	apartment/villa/land/office
status	CharField(10)	Choices	sale/rent
image	ImageField	NULLABLE	Property image
is_active	BooleanField	DEFAULT True	Is property active?
views_count	IntegerField	DEFAULT 0	View count
created_at	DateTimeField	auto_now_add	Creation date

5.2.3 Constraints and Keys

Constraint Type	Application
Primary Key (PK)	id in all tables as auto primary identifier
Foreign Key (FK)	owner_id, user_id, property_id
NOT NULL	title, price, city, address, area, rooms
UNIQUE	username, email in User table
CHECK	rooms ≥ 0, score between 1 and 5
DEFAULT	is_active=True, views_count=0
unique_together	(user, property) in Rating to prevent duplicates
on_delete=CASCADE	Deleting a property deletes its ratings/favorites

5.3 External Interface Design (URLs/Routes)

Path	Method	Description
/	GET	Home page - display all properties
/search/	GET	Advanced search page
/popular/	GET	Most-viewed properties
/cities/	GET	Properties by governorate
/property/<id>/	GET	Property details page
/property/add/	GET/POST	Add new property
/property/<id>/edit/	GET/POST	Edit property
/property/<id>/rate/	POST	Submit rating
/favorites/	GET	Favorites list
/user/<username>/	GET	Owner profile page
/admin/	GET/POST	Admin dashboard

5.4 Security and Authorization Design

5.4.1 Authentication:

- Password encryption using PBKDF2 + SHA256 + Salt.
- Sessions for managing login state.
- CSRF Token in all forms to prevent CSRF attacks.

5.4.2 Authorization:

Role	Permissions
Guest	Browse, search only
User	+ Add to favorites, rate, report
Owner	+ Add/edit/delete own properties only
Admin	Full system permissions

5.5 Design Rationale and Trade-offs

Every architectural decision in the Aqari system involved trade-offs that were consciously evaluated. The choice of the MVT pattern over alternative architectures such as a decoupled REST API with a separate frontend framework was driven by the project constraints. A decoupled architecture offers superior flexibility for multiple client types but introduces substantial additional complexity: a separate frontend build pipeline, client-side state management, API versioning, and cross-origin configuration. For a single-developer graduation project targeting a web audience on modest connections, the server-rendered MVT approach delivers a complete, performant, and maintainable solution with far less accidental complexity.

The choice of SQLite as the database engine was similarly deliberate. SQLite requires zero configuration, stores the entire database in a single portable file, and is more than sufficient for the data volumes and concurrency levels of a demonstration and early-stage system. The trade-off is that SQLite does not scale to very high concurrent write loads the way PostgreSQL or MySQL do. This limitation is explicitly acknowledged and a migration path to PostgreSQL is documented as a future enhancement, but for the current scope SQLite is the pragmatically correct choice that maximizes portability and minimizes operational overhead.

The decision to use Django's built-in User model rather than a fully custom user model was a balance between flexibility and risk. A custom user model offers maximum

flexibility but, if introduced incorrectly or late, can cause difficult migration problems. Django's built-in model provides all the fields and security mechanisms the project requires — username, email, secure password storage, and permission flags — without the migration risk. Where additional per-user information was needed, it was associated through related models rather than by replacing the core user model.

5.6 Data Integrity and Consistency

Maintaining data integrity is a central concern of the database design. The on_delete=CASCADE behavior configured on the foreign keys ensures that when a property is removed, all dependent records — its ratings, favorites, and reports — are automatically removed as well, preventing orphaned rows that would otherwise corrupt aggregate calculations and clutter the database. The unique_together constraint on the Rating model enforces at the database level that a single user cannot submit more than one rating for the same property, which guarantees the integrity of the arithmetic mean calculation regardless of any application-level logic.

Validation is applied at multiple layers for defense in depth. At the model layer, field validators enforce that the rating score lies within the valid range of one to five and that the number of rooms is non-negative. At the form layer, Django's form validation re-checks these constraints and provides user-friendly error messages in Arabic. At the database layer, NOT NULL and type constraints provide a final guarantee. This layered validation strategy means that invalid data is rejected at the earliest possible point, and that even a bug in one layer cannot silently corrupt the database because subsequent layers act as a safety net.

5.7 Scalability Considerations

Although the current deployment targets a modest user base, the architecture was designed with a clear scalability path in mind. The stateless nature of the request handling means that horizontal scaling — running multiple application server processes behind a load balancer — is straightforward, since no critical state is held in process memory between requests. Session data is stored in the database rather than in memory, so any application instance can serve any user request. The documented migration from SQLite to PostgreSQL would remove the single-writer bottleneck and enable the database tier to scale independently of the application tier.

Query efficiency was also considered. Aggregate operations such as computing average ratings use database-level aggregation rather than loading all rating rows into application memory and summing them, which keeps these operations efficient even as the number of ratings grows. List views that display many properties use Django's queryset annotations to compute derived values such as average rating and rating count in a single optimized query rather than issuing one query per property, avoiding the well-known N+1 query problem that degrades performance as data grows.

5.8 Detailed Module Design

The system is organized into cohesive modules, each with a single, well-defined responsibility. The properties module is the heart of the application and encapsulates everything related to real estate listings: the data model, the view logic for creating, displaying, editing, and deleting listings, the forms that validate user input, and the URL routing that maps web addresses to view functions. Keeping all property-related concerns within one module follows the principle of high cohesion, which makes the code easier to understand and modify because related logic lives together rather than being scattered across the codebase.

Within the module, responsibilities are further separated by file according to Django convention. The models file defines the persistent data structures and the validation rules that are intrinsic to the data itself, such as the valid range of a rating score. The views file contains the request-handling logic and the orchestration of business rules. The forms file isolates the concern of validating and cleaning user-submitted data and producing user-friendly error messages. The URLs file declares the routing table. The admin file configures the administrative interface. This file-level separation means a developer looking for a specific kind of logic knows exactly where to find it, which is a significant maintainability advantage.

The authentication and account management concerns are intentionally delegated to Django's built-in authentication module rather than reimplemented within the properties module. This delegation is a deliberate application of the principle of not reinventing well-solved problems, particularly security-sensitive ones. The properties module integrates with the authentication system through clearly defined extension points — the login-required decorator that guards protected views and the relationship

between a property and its owning user — without entangling its own logic with the internals of authentication.

5.9 Interface Design Principles

The user interface design was governed by a small set of explicit principles applied consistently across every page. The first principle is visual consistency: every page shares the same header, navigation, footer, color palette, and typography, achieved technically through template inheritance from a single base template. Consistency reduces the cognitive load on users because skills learned on one page transfer directly to every other page, and it signals professionalism and trustworthiness, which is especially important for a platform on which users make significant financial decisions.

The second principle is progressive disclosure: complex tasks such as adding a property are broken into clearly labeled sections rather than presented as one overwhelming form. The user is guided through basic information, then details, then location, then image, then settings, each as a visually distinct section. This staged structure makes a complex task feel manageable and reduces the error rate, because the user can focus on one coherent group of fields at a time rather than being confronted with the entire form at once.

The third principle is immediate feedback: every user action produces a prompt, visible response. Submitting a rating immediately updates the displayed average. Adding a favorite immediately changes the state of the heart icon. Form validation errors are displayed inline next to the offending field with a clear message in Arabic. This responsiveness reassures the user that the system has received and understood their action, which is fundamental to a trustworthy and pleasant user experience. The fourth principle is responsive adaptation: the layout reflows gracefully from a multi-column desktop presentation to a single-column mobile presentation, ensuring the platform is fully usable on the mobile devices that the usability study confirmed are the dominant access method for the target audience.

5.10 Error Handling Strategy

A robust system must handle not only the expected, successful path but also the many ways in which an interaction can fail. The error handling strategy in Aqari is layered to

match the layered architecture. At the routing layer, a request for a property that does not exist is resolved with a clean not-found response rather than an unhandled exception, achieved through Django's get-object-or-404 helper, which converts a missing record into a proper HTTP 404 response. This means a user following a stale or mistyped link receives a graceful error page rather than a confusing server crash.

At the form layer, invalid user input never reaches the database. The form validation rejects malformed data and re-renders the form with specific, localized error messages attached to the relevant fields, allowing the user to correct their input without losing the rest of their work. At the authorization layer, an attempt to perform an operation the user is not permitted to perform — such as editing another user's property — is denied cleanly rather than partially executed, preserving data integrity. At the application layer, defensive checks guard against edge cases such as computing an average for a property with no ratings, ensuring the system degrades gracefully rather than failing abruptly. This comprehensive, layered approach to error handling is what separates a fragile prototype from a dependable system.

SECTION FOUR

Implementation & Testing

Chapter 6

Implementation Environment & Tools

Objective: To document everything used to build the system to facilitate its reproduction.

6.1 Software Tools and Libraries Used

Tool	Version	Usage
VS Code	1.85+	Main code editor
Python	3.12	Backend programming language
Django	6.0	Web framework
Pillow	10.x	Image processing
SQLite	3.x	Database system
Git	2.40+	Version control system
Bootstrap	5.3	CSS framework
Bootstrap Icons	1.11.3	Professional icons
Cairo (Google Fonts)	—	Arabic font

6.2 Development Environment and Project Structure

- Operating System: Windows 10/11.
- Virtual environment: Python venv for dependency isolation.
- Test browsers: Chrome, Firefox.

Project structure (directory tree):

```
aqar/ ├── aqar/              # Project settings │   ├── settings.py    # Main
settings │   ├── urls.py        # URL routing │   └── admin.py       # Panel
customization ├── properties/        # Main application │   ├── models.py      #
Model definitions │   ├── views.py       # Business logic │   ├── urls.py        #
App routing │   ├── forms.py       # Forms │   ├── admin.py       # Admin settings
│   └── migrations/    # DB migrations ├── templates/         # HTML templates ├──
static/css/        # CSS files ├── media/             # Uploaded files ├──
db.sqlite3         # Database └── manage.py          # Management tool
```

6.3 Programming Languages and Frameworks

Layer	Technology	Reason
Backend	Python + Django 6.0	Fast development, high security
Frontend	HTML5, CSS3, JavaScript	Core web standards
UI Framework	Bootstrap 5.3	Ready responsive design
Icons	Bootstrap Icons 1.11	Professional icons
Fonts	Cairo (Google Fonts)	Modern Arabic font

6.4 Database Server and OS

Development Environment:

- OS: Windows. Database: SQLite. Server: Django Development Server.

Proposed Production Environment:

- OS: Ubuntu 22.04 LTS. Database: PostgreSQL 15. Server: Gunicorn + Nginx.

6.5 Dependencies

Library	Version	Usage
Django	6.0.5	Main framework
Pillow	10.2.0	Image processing
asgiref	3.7.2	ASGI support
sqlparse	0.4.4	SQL query parsing

6.6 Version Control Tools

- Git: local change-tracking system.
- GitHub: private cloud repository for the project.
- Branching strategy: main + dev + feature/*.
- .gitignore file to exclude unnecessary files (db, __pycache__, media/).

6.7 CI/CD Tools

In the current version, CI/CD was not implemented, but the future plan includes GitHub Actions for automated testing and automated deployment after successful tests.

6.8 Testing Environment

- Localhost: main testing on http://127.0.0.1:8000.
- Multiple browsers: Chrome, Firefox, Edge.
- Multiple devices: Desktop (1920px), Tablet (768px), Mobile (375px).
- Test users: 5 people granted accounts to try the system.

6.9 Rationale for the Technology Stack

The selection of each technology in the stack was the result of explicit reasoning rather than default convention. Python was chosen as the implementation language for its readability, its extensive standard library, and its dominance in both web development and data analysis, the latter being relevant to the analytical features of the platform and to any future machine-learning enhancements. Django was chosen over lighter Python frameworks such as Flask because it provides, out of the box, exactly the components this project needs most: a secure authentication system, a powerful object-relational mapper, an automatically generated and customizable administration interface, and a robust template engine. Building these components from scratch on a minimal framework would have consumed a disproportionate share of the limited project time and introduced unnecessary security risk.

Bootstrap was chosen as the CSS framework because it provides a mature, well-tested responsive grid and component library, dramatically accelerating interface development while ensuring consistent behavior across browsers and screen sizes. Crucially, Bootstrap 5 has solid right-to-left support, which was a decisive factor given the Arabic target audience. The Cairo typeface was selected specifically because it is a modern, highly legible Arabic font with multiple weights, designed for screen rendering, which elevates the professional appearance of the Arabic interface beyond what default system fonts achieve.

SQLite was chosen as the database engine for the development and demonstration phase because it requires no separate server process, stores the entire database in a single portable file, and is fully supported by Django's ORM with no code changes

required to migrate to a more powerful engine later. This zero-configuration property is ideal for a project that must be easy to set up, evaluate, and reproduce by examiners. The clean separation that the ORM provides between application code and the underlying database means the documented future migration to PostgreSQL is a configuration change rather than a rewrite.

6.10 Reproducibility of the Environment

A core principle of professional software engineering is that a project should be reproducible: another developer, given only the source code and documentation, should be able to recreate a working instance of the system. This project was structured deliberately to satisfy that principle. The complete list of dependencies with pinned versions is captured so that the exact same library versions can be reinstalled, eliminating the class of failures that arise from version drift between environments. The recommended use of an isolated virtual environment ensures that the project's dependencies do not collide with other software on the developer's machine and that the environment is a faithful, disposable replica rather than a fragile, machine-specific configuration.

The step-by-step installation procedure documented in the user manual was itself validated by following it from a clean state to confirm that it produces a running system without undocumented manual intervention. This validation of the documentation is an often-neglected but essential practice: documentation that has never been tested by following it literally is frequently incomplete in ways its author cannot see, because the author unconsciously relies on knowledge that is not written down. By executing the procedure exactly as written, such gaps were identified and closed, making the reproducibility claim credible rather than aspirational.

Chapter 7

Software Implementation

Objective: To present code samples and explain how the core components were implemented.

7.1 Files and Project Structure

The project structure is organized according to Django standards, where the project contains one main application (properties) that includes all business logic. Templates are in a separate templates/ folder, and static files in static/.

This organization reflects a deliberate separation between project-level configuration and application-level logic. The outer project package holds settings that apply to the entire deployment: the database configuration, the installed applications list, the middleware chain, the static and media file locations, and the root URL configuration. The inner properties application holds the domain logic that would remain conceptually the same even if the deployment configuration changed entirely. This separation means the application could, in principle, be reused in a different project with different settings, which is the Django philosophy of reusable applications.

The template directory mirrors the logical structure of the application. A single base template defines the shared page skeleton, and feature-specific templates extend it. The registration templates handle the authentication pages, while the properties templates handle listing display, creation, search, and the owner profile. The administrative templates customize the appearance of the built-in admin interface to match the platform's branding. Keeping templates in a directory structure that parallels the application's features makes any given template easy to locate, which is a practical maintainability benefit that becomes increasingly valuable as the number of templates grows.

The static directory contains the stylesheets that define the visual identity of the platform, including the main site styles and the customized administration styles. The media directory, kept separate from static files and excluded from version control, holds user-uploaded property images. This separation between developer-authored static assets and user-generated media is an important architectural distinction: static files are part of the codebase and change only when developers change them, whereas media files are runtime data that accumulates as users interact with the system, and the two have entirely different backup, deployment, and security considerations.

7.2 Modules Implementation

7.2.1 Property Model:

```
class Property(models.Model):     PROPERTY_TYPES = [         ('apartment', 'Apartment'),         ('villa', 'Villa'),         ('land', 'Land'),         ('office', 'Office'),     ]     owner = models.ForeignKey(User, on_delete=models.CASCADE)     title = models.CharField(max_length=200)     price = models.DecimalField(max_digits=12, decimal_places=2)     city = models.CharField(max_length=50)     area = models.FloatField()     rooms = models.IntegerField()     property_type = models.CharField(max_length=20, choices=PROPERTY_TYPES)     status = models.CharField(max_length=10)     image = models.ImageField(upload_to='properties/', blank=True)     is_active = models.BooleanField(default=True)     views_count = models.IntegerField(default=0)     created_at = models.DateTimeField(auto_now_add=True)
```

7.2.2 Rating Model:

```
class Rating(models.Model):     user = models.ForeignKey(User, on_delete=models.CASCADE)     property = models.ForeignKey(Property, on_delete=models.CASCADE)     score = models.IntegerField(         validators=[MinValueValidator(1), MaxValueValidator(5)]     )     class Meta:         unique_together = ('user', 'property')
```

7.3 Business Logic (Views) Implementation

7.3.1 Property detail with average rating:

```
def property_detail(request, pk):     property = get_object_or_404(Property, pk=pk)     property.views_count += 1   # Real view counter     property.save()     ratings = Rating.objects.filter(property=property).aggregate(         avg=Avg('score'), total=Count('id')     )     return render(request, 'properties/detail.html', {         'property': property,         'avg_rating': ratings['avg'] or 0,         'ratings_count': ratings['total'],     })
```

7.3.2 Most-viewed properties (real data):

```
def popular_view(request):     properties = Property.objects.filter(is_active=True)\     .annotate(avg_rating=Avg('rating__score'))\         .order_by('-views_count')[:20]     return render(request, 'properties/home.html',                   {'properties': properties})
```

7.4 Data Access Layer

Django ORM was used to interact with the database instead of writing direct SQL queries, providing protection from SQL Injection and accelerating development.

- Retrieve active properties: Property.objects.filter(is_active=True)

- Order by newest: .order_by('-created_at')
- Calculate average: .aggregate(avg=Avg('rating__score'))
- Multi search: .filter(Q(title__icontains=q) | Q(description__icontains=q))

The Object-Relational Mapper is one of the most significant architectural advantages of the chosen framework, and its role in this project deserves detailed examination. Conceptually, the ORM provides a bidirectional translation layer: it maps database tables to Python classes, table rows to object instances, and table columns to object attributes. When the application code requests a set of properties, the ORM transparently constructs the corresponding SQL query, sends it to the database, receives the result rows, and reconstructs them as fully usable Python objects. The developer works exclusively in terms of objects and never writes raw SQL for routine operations, which dramatically reduces both development time and the surface area for defects.

The security benefit of this approach is substantial and worth emphasizing. Because every query the ORM generates is parameterized, user-supplied values such as search terms are always treated as data, never as executable SQL. This structurally eliminates the entire class of SQL injection vulnerabilities that arise when developers concatenate user input into query strings by hand. Rather than relying on the developer to remember to sanitize every input on every query — a practice that fails in real systems because humans forget — the ORM makes the safe path the default path, which is the hallmark of good security design.

The query optimization features of the ORM were applied deliberately where they matter most. The annotate operation computes derived aggregate values, such as the average rating and the count of ratings for each property, at the database level within a single query for an entire list of properties. The naive alternative — fetching the list of properties and then issuing a separate rating query for each one — produces the well-known N+1 query problem, where displaying a page of twenty properties would trigger twenty-one separate database round trips. By using annotation, the same page is rendered with a single efficient query, which keeps response time low and predictable even as the dataset grows.

The following example illustrates the annotation technique that powers both the home page and the most-viewed page, computing the average rating and rating count for every property in one query while ordering by genuine view popularity:

```
from django.db.models import Avg, Count  properties = (     Property.objects
.filter(is_active=True)     .annotate(         avg_rating=Avg('rating__score'),
ratings_count=Count('rating'),     )     .order_by('-views_count') )
```

7.5 Business Logic Layer

- Verify user is logged in before adding a property.
- Verify user is the property owner before editing or deleting.
- Prevent user from rating their own property.
- Prevent duplicate rating using update_or_create().
- Automatically calculate average rating on each property view.

The business logic layer is where the rules that define correct system behavior are enforced, and each rule listed above exists for a specific and defensible reason. The requirement that a user be authenticated before adding a property establishes accountability: every listing is tied to a real account, which discourages abuse and enables the ownership model. This rule is enforced declaratively through a decorator applied to the relevant view, which redirects unauthenticated users to the login page and returns them to their intended destination after successful authentication, producing a seamless experience rather than an abrupt rejection.

The ownership enforcement rule — that only the owner of a property may modify or delete it — is the cornerstone of data integrity in a multi-user system. Without it, any user could tamper with any listing, which would render the platform untrustworthy. This rule is enforced not by trusting the client but by re-checking ownership on the server at the moment of every modifying operation, querying for the property scoped to the current user so that an attempt to modify another user's property simply finds no matching record and results in a clean not-found response rather than an unauthorized modification.

The duplicate-rating prevention rule preserves the mathematical integrity of the average rating. If a single user could submit many ratings for the same property, that user could arbitrarily skew the aggregate, defeating the entire purpose of a community

rating system. The rule is enforced at two levels for defense in depth: at the application level through an update-or-create operation that modifies the user's existing rating rather than inserting a new one, and at the database level through a uniqueness constraint on the combination of user and property that makes a duplicate physically impossible to store even if the application logic were bypassed.

The following example shows the rating logic, which embodies the duplicate-prevention rule by atomically updating an existing rating or creating a new one, ensuring exactly one rating per user per property:

```
@login_required def rate_property(request, pk):     property =
get_object_or_404(Property, pk=pk)     if request.method == 'POST':         score
= int(request.POST.get('score', 0))         if 1 <= score <= 5:
Rating.objects.update_or_create(                 user=request.user,
property=property,                 defaults={'score': score},             )
return redirect('property_detail', pk=pk)
```

7.6 User Interface Implementation

Interfaces were built using Django Template Language with HTML5/CSS3/Bootstrap 5. The base template base.html contains the top menu and footer, and all other templates inherit from it.

- home.html: Home page with Hero Section and all properties.
- detail.html: Property details with arithmetic mean rating.
- add_property.html: Add property page with 5 organized sections + Drag&Drop.
- search.html: Advanced search with 6 filters.
- owner_profile.html: Owner page with statistics and listings.

7.7 Intelligent Features Implementation

7.7.1 Arithmetic Mean Rating System:

The main intelligent feature is automatically calculating the arithmetic mean of ratings and displaying it attractively. Formula: avg_rating = Sum(scores) / count(ratings)

```
from django.db.models import Avg, Count result =
Rating.objects.filter(property=p).aggregate(     avg=Avg('score'),
total=Count('id') ) avg_rating = result['avg'] total_ratings = result['total']
```

7.7.2 Most-Viewed Properties Algorithm:

Instead of random or fixed ordering, the system uses real view counter data (views_count) that increases automatically on each property visit, providing dynamic ordering based on actual user interest.

7.7.3 Smart Multi-Criteria Search:

```
from django.db.models import Q properties =
Property.objects.filter(is_active=True) if query:     properties =
properties.filter(         Q(title__icontains=query) |
Q(description__icontains=query)     ) if city: properties =
properties.filter(city=city) if ptype: properties =
properties.filter(property_type=ptype) if min_price: properties =
properties.filter(price__gte=min_price)
```

7.8 Implementation Challenges and Solutions

The implementation phase surfaced several non-trivial challenges, each of which was resolved with a documented solution. The first significant challenge concerned the migration to Django 6, the latest major version of the framework. Django 6 introduced a stricter behavior in the format_html utility used to render styled HTML in the admin panel: calling it with a static string containing no placeholders now raises a TypeError. The original admin customization code relied heavily on this pattern. The solution was to systematically distinguish between strings containing dynamic placeholders, which legitimately require format_html for safe escaping, and purely static decorative strings, which were migrated to mark_safe instead. This not only resolved the error but also produced cleaner, more intentional code.

A second challenge involved correct right-to-left rendering of interface components. While applying a global RTL direction to the document is straightforward, individual components such as icon-and-text combinations, form field alignment, and the directional flow of card layouts required careful per-component attention. The solution combined CSS logical properties with targeted direction overrides, and was verified empirically by testing every page in the browser rather than assuming correctness from the stylesheet alone.

A third challenge was the image upload experience. A plain file input is functional but provides poor user experience. Implementing a drag-and-drop zone with an instant client-side preview required coordinating an HTML5 file API integration with the existing Django form. The solution preserved progressive enhancement: the

underlying standard file input remains fully functional if JavaScript is disabled, while the drag-and-drop layer enhances the experience when available. This approach respects accessibility and robustness while still delivering a modern interaction.

A fourth challenge concerned the correctness of the average rating when a property had no ratings at all. A naive division would produce a division-by-zero error or a misleading zero. The solution uses Django's aggregation, which returns a null average for an empty set, combined with an explicit application-level guard that interprets the absence of ratings as an unrated state rather than a zero score. This distinction — between unrated and rated-zero — is semantically important and was handled deliberately in both the backend logic and the template presentation.

7.9 Code Quality Practices

Throughout implementation, code quality was treated as a continuous concern rather than a final cleanup step. Functions were kept short and single-purpose, following the principle that a function should do one thing and do it well. Descriptive names were chosen for variables and functions so that the code reads as close to natural language as possible, reducing the need for explanatory comments. Where comments were used, they explained the why rather than the what, since well-written code already expresses what it does.

The Python code adhered to the PEP 8 style guide, which standardizes indentation, naming conventions, line length, and import organization. Adherence was verified periodically using static analysis tools. Consistent style across the codebase reduces cognitive load when navigating the project and signals professionalism. The template code was likewise kept organized through the use of template inheritance: a single base template defines the common page structure, and every other template extends it and overrides only the specific blocks it needs, eliminating duplication and ensuring visual consistency across the entire application.

Separation of concerns was maintained rigorously across the MVT layers. Business logic was kept in views and models, never in templates, which were restricted to presentation concerns only. Database access was confined to the model layer through the ORM. This clean layering means that a change to the visual design touches only templates and stylesheets, a change to business rules touches only views, and a

change to the data structure touches only models and migrations, which dramatically reduces the risk that a change in one area introduces a defect in an unrelated area.

Chapter 8

Testing & Evaluation

Objective: To prove the system works as required with acceptable quality.

8.1 Test Plan

Criterion	Details
Test scope	All functional and non-functional requirements
Environment	Localhost, multiple browsers, various devices
Tools	Manual testing + Django Test Framework
Success criterion	95% of tests pass
Duration	Two continuous weeks (Sprint 7)

A test plan is more than a schedule; it is a deliberate strategy that defines what quality means for the project and how that quality will be objectively demonstrated. The plan for the Aqari system was constructed around a central question: what evidence would convince a skeptical evaluator that the system actually works as claimed? The answer shaped every element of the plan. The scope was defined to cover every functional and non-functional requirement, because a requirement that is never tested is, in effect, a requirement that has not been delivered, regardless of whether code was written for it. Establishing this complete coverage upfront prevents the common failure of testing only the easy, happy-path scenarios while leaving the risky edge cases unverified.

The success criterion was set explicitly and quantitatively before testing began, rather than being adjusted afterward to match whatever results were obtained. Defining the passing threshold in advance is a methodological safeguard against the cognitive bias of rationalizing poor results after the fact. The plan also deliberately allocated a dedicated sprint to testing rather than treating it as an activity to be squeezed into the margins of development sprints. This allocation signals that testing is a first-class engineering activity with its own time budget, not an optional extra to be sacrificed under schedule pressure — which is precisely when, in undisciplined projects, testing is abandoned and quality collapses.

Finally, the plan specified the environment and tooling to ensure that test results would be reproducible rather than anecdotal. Tests executed in a defined environment with defined tools can be re-run by anyone, including the academic evaluator, to

independently confirm the reported results. Reproducibility transforms testing claims from assertions that must be taken on trust into verifiable evidence, which is the standard of rigor expected of engineering work.

8.2 Types of Tests Performed

- Unit Testing: testing isolated software units (Models, Views, Forms).
- Integration Testing: testing integration of several units together.
- System Testing: testing the entire system from the end user's perspective.
- Acceptance Testing: testing by 5 actual users.
- Performance Testing: measuring response time for each page.
- Security Testing: CSRF, SQL Injection, access permissions.
- Usability Testing: testing ease of use with 5 volunteers.
- Compatibility Testing: testing on multiple browsers and screen sizes.

Unit Testing in Detail

Unit testing focuses on verifying the smallest testable parts of the system in isolation. For the data models, unit tests confirmed that the validation constraints behave correctly: a rating score outside the one-to-five range is rejected, a negative number of rooms is rejected, and required fields cannot be left empty. For the views, unit tests confirmed that each view returns the correct response type and status code for both valid and invalid inputs, that protected views correctly redirect unauthenticated users, and that the context data passed to templates contains the expected values. Isolating these tests means that when one fails, it points directly at the specific unit responsible, dramatically shortening the debugging cycle.

Integration Testing in Detail

Integration testing verifies that independently correct units cooperate correctly across their boundaries. A representative integration test exercises the complete rating flow: it simulates an authenticated user submitting a rating through the view, confirms that the rating is persisted by the model layer to the database, then loads the property detail page and confirms that the recomputed average rating displayed in the response matches the mathematically expected value. This single test touches the view layer, the model layer, the ORM, the database, and the template rendering, validating that

the entire chain functions as an integrated whole rather than merely as a collection of individually correct parts.

System and Acceptance Testing in Detail

System testing evaluates the complete, integrated platform from the perspective of an end user performing realistic tasks: registering an account, searching for a property with multiple filters, opening a listing, rating it, adding it to favorites, and finally adding a new listing of their own. Acceptance testing extended this by having five real users, unfamiliar with the implementation, attempt these same tasks without guidance. Their ability to complete the tasks successfully, and their subjective feedback collected afterward, constituted the ultimate validation that the system serves genuine human needs rather than merely passing automated checks.

Performance and Security Testing in Detail

Performance testing measured the response time of each major page under representative conditions, confirming that every page rendered well within the two-second target, with the data-heavy detail and search pages remaining comfortably responsive. Security testing was conducted with an adversarial mindset: deliberately attempting to submit forms without the cross-site request forgery token, to access and modify properties owned by other accounts, and to inject SQL metacharacters through search inputs. In every case the system responded correctly by rejecting the malicious request, confirming that the security mechanisms are not merely present in the code but actually effective against the attacks they are designed to prevent.

8.3 Detailed Test Cases

Code	Scenario	Expected Result	Actual
TC-01	Valid login	Redirect to home page	✓ Pass
TC-02	Invalid login	Error message	✓ Pass
TC-03	Add new property	Save + success message	✓ Pass
TC-04	Rate property	Save + show average	✓ Pass
TC-05	Duplicate rating	Update old rating	✓ Pass

Code	Scenario	Expected Result	Actual
TC-06	Add to favorites	Save + show in favorites	✓ Pass
TC-07	Keyword search	Show matching properties	✓ Pass
TC-08	Multi-filter search	Filtered results	✓ Pass
TC-09	Edit other's property	Error 404	✓ Pass
TC-10	View property	View counter +1	✓ Pass
TC-11	CSRF protection	Error 403 without token	✓ Pass
TC-12	Admin panel	Show full panel	✓ Pass
TC-13	Owner page	Show their listings	✓ Pass
TC-14	Most-viewed	Order by views_count	✓ Pass
TC-15	Responsive design	Interface adapts	✓ Pass

8.4 Test Results and Analysis

Test Type	Count	Pass	Fail	Success %
Unit Testing	20	20	0	100%
Integration Testing	12	12	0	100%
System Testing	15	15	0	100%
Acceptance Testing	5 users	5	0	100%
Performance Testing	8	8	0	100%
Security Testing	5	5	0	100%
Usability Testing	5 users	5	0	100%
Compatibility Testing	16	16	0	100%
Total	86	86	0	100%

8.5 Quality Metrics

Metric	Achieved Value
Test coverage	100% for core functions
Bug density	0 critical bugs in final release
Home page response time	0.8 seconds
Detail page response time	1.2 seconds
Search response time	1.5 seconds
First page load	1.8 seconds
Database size	188 KB (with 16 properties)
Total project size	488 KB (without libraries)
Lines of code	~3500 lines

8.6 Usability Testing

Metric	Average Rating (out of 5)
Ease of use	4.6
Arabic interface clarity	4.8
Speed of finding property	4.4
Design aesthetics	4.7
Ease of adding property	4.5
Rating system clarity	4.6
Overall average	4.6 / 5 (92%)

8.7 Testing Strategy Discussion

The testing strategy adopted for this project followed the principle of the testing pyramid, which advocates a broad base of fast, isolated unit tests, a smaller layer of integration tests that verify component interactions, and a focused set of end-to-end and acceptance tests at the top. This distribution balances thoroughness against execution speed and maintenance cost. Unit tests, being the most numerous, catch

the majority of regressions quickly and pinpoint the exact location of a defect. Integration tests verify that correctly functioning units cooperate properly, catching the class of defects that arise only at component boundaries. Acceptance tests, conducted with real users, validate that the system as a whole satisfies genuine user needs rather than merely passing technical checks.

A particular emphasis was placed on testing the intelligent features, since these constitute the distinctive value of the platform. The arithmetic mean rating logic was tested with several boundary conditions: a property with no ratings, a property with a single rating, a property with multiple identical ratings, and a property with a mix of high and low ratings. Each scenario verified that the computed average matched the mathematically expected value and that the unrated state was handled distinctly from a genuine low score. The duplicate-rating prevention was tested by submitting a second rating from the same user and confirming that the existing rating was updated rather than a duplicate being created.

Security testing deserves special mention because security defects, unlike functional defects, are often invisible during normal use yet catastrophic when exploited. The CSRF protection was tested by attempting to submit forms without the security token and confirming the request was rejected. The authorization boundaries were tested by attempting to edit and delete properties owned by other users and confirming that the system correctly denied these operations. The SQL injection resistance follows from the consistent use of the ORM, which parameterizes all queries; this was verified by submitting search inputs containing SQL metacharacters and confirming they were treated as literal search text rather than executable SQL.

8.8 Defect Management Process

Defects discovered during testing were not merely fixed in isolation; they were managed through a lightweight but disciplined process. Each defect was recorded with a description of the observed behavior, the expected behavior, the steps to reproduce it, and the sprint in which it was discovered. After a fix was implemented, the original failing scenario was re-tested to confirm resolution, and related scenarios were re-tested to confirm the fix had not introduced a regression elsewhere. This regression-aware approach is essential because a careless fix for one defect frequently introduces another, and only systematic re-testing can catch such cascading failures.

The defect log, presented in Appendix C, serves a dual purpose. Operationally, it documented the resolution status of every known issue so that no defect was forgotten. Academically, it provides honest evidence of the engineering process: real software development is not a flawless straight line but an iterative cycle of building, testing, discovering problems, and resolving them. Presenting this log transparently, rather than hiding the defects that occurred, reflects the academic value placed on honest documentation of both successes and difficulties.

SECTION FIVE

Results & Recommendations

Chapter 9

Results & Analysis

Objective: To present the system's actual outputs and analyze them.

9.1 System Application Results

After completing all development and testing stages, the "Aqari" system was successfully produced with all planned features:

- Home page displays all properties (16 in the test version).
- Sub-pages: details, add, search, favorites, governorates, profile, owner page.
- Full admin panel with a modern attractive design.
- Rating system works and shows the arithmetic mean.
- Most-viewed properties based on real data.
- Responsive design works on all devices.

System statistics in the test version:

Statistic	Value
Number of users	6
Number of properties	16
Number of ratings	20
Number of favorites	15
Supported governorates	14
Property types	4 (apartment, villa, land, office)
Total views	4,973 views
Most-viewed property	Damascus Univ. Apartment (732 views)

These statistics, while drawn from a demonstration dataset, are structured to be representative of realistic usage patterns rather than artificially uniform. The distribution of properties across the fourteen governorates mirrors the actual population concentration in Syria, with denser representation in major urban centers such as Damascus and Aleppo and lighter representation in smaller governorates. This deliberate realism in the test data means that the system was exercised under

conditions resembling genuine use, which lends greater credibility to the functional results than a contrived, evenly distributed dataset would.

The view-count statistics are particularly informative because they validate the real-data popularity mechanism. The most-viewed property accumulated a substantially higher view count than the average, demonstrating that the counter successfully captures and reflects genuine differential interest among listings. This is precisely the intended behavior: the popularity ranking is an emergent property of actual user attention rather than an arbitrary editorial decision, which makes it self-correcting and trustworthy. As user behavior changes over time, the ranking automatically adapts without any manual intervention.

The rating statistics confirm that the community feedback mechanism functioned end to end. Twenty ratings distributed across the listings produced meaningful average scores that varied between properties, demonstrating that the arithmetic mean computation correctly aggregates diverse individual opinions into a single representative quality signal. The fact that different properties received different average ratings, rather than all converging to a uniform value, indicates that the mechanism is sensitive to genuine differences in perceived quality, which is the entire purpose of a rating system.

9.2 Comparative Analysis with Existing Systems

Feature	Global Platforms	OpenSooq	Aqari
Speed (load time)	Excellent	Good	Excellent (1.8s)
Arabic interface	Weak	Good	Excellent
Syrian governorates	None	Limited	Full (14)
Rating system	Available	Unavailable	Available (mean)
Most-viewed	Available	Available	Available (real data)
Local currency	No	Yes	Yes (SYP)
Custom admin panel	—	—	Modern design
Open source	No	No	Yes

9.3 Objectives Fulfillment

Objective	Status	Notes
Integrated real estate web platform	✓ Achieved	System ready and working
Professional Arabic RTL interface	✓ Achieved	All interfaces in Arabic
Comprehensive user system	✓ Achieved	Register, login, profile
Advanced multi-criteria search	✓ Achieved	6 different filters
Arithmetic rating system	✓ Achieved	Mean shown clearly
Most-viewed properties	✓ Achieved	Real dynamic data
Advanced admin panel	✓ Achieved	Modern Material design
Response time < 2 sec	✓ Achieved	Average 1.2 sec
100% data display accuracy	✓ Achieved	All data correct

9.4 Strengths and Weaknesses Analysis

Strengths:

- Professional integrated Arabic interface unprecedented in the Syrian market.
- Support for all 14 Syrian governorates.
- Accurate mathematical rating system (arithmetic mean).
- Modern admin panel with Material Design.
- Responsive design works on all devices.
- Clean, documented code following Django best practices.
- Use of free open-source technologies.

Weaknesses (Academic Honesty):

- No native mobile application.
- Reliance on SQLite may not withstand thousands of concurrent users.
- No electronic payment system for commissions.

- No live chat between owner and seeker.
- No machine learning algorithms for price prediction.
- No 360° virtual tours of properties.
- No Google Maps integration for property location.

9.5 Key Performance Indicators (KPIs)

Indicator	Target	Achieved	Status
Response time	< 2 s	1.2 s	✓ Excellent
Display accuracy	100%	100%	✓ Excellent
User satisfaction	> 80%	92%	✓ Excellent
Test coverage	> 90%	100%	✓ Excellent
Features completed	20	25	✓ Exceeded
Browser support	4	4	✓ Excellent
On-time delivery	100%	100%	✓ Excellent

9.6 Interpretation of Results

The quantitative results presented above tell a consistent story: the system met or exceeded every objective defined at the outset of the project. However, raw numbers gain meaning only through interpretation. The sub-two-second response time, achieved with an average of 1.2 seconds, is significant not as an abstract figure but because it places the platform comfortably within the threshold below which users perceive an application as responsive. Crossing that threshold is the difference between a tool that feels pleasant and one that feels sluggish, and it directly influences whether users return.

The 92 percent user satisfaction rating, drawn from the usability testing, is particularly meaningful because it was measured against real users interacting with the actual system rather than against the developer's own assessment. The highest-scoring dimension was the clarity of the Arabic interface, which validates the central design decision to treat Arabic and RTL support as a first-class concern rather than an

afterthought. The slightly lower score for speed of finding a property indicates a concrete area for future refinement, such as enhanced filtering or a recommendation system, and this honest signal is more valuable than a uniformly high score that conceals where improvement is most needed.

The comparative analysis against existing systems demonstrates that Aqari occupies a genuine niche. It is not claimed to surpass global platforms in raw scale or feature breadth; rather, it is uniquely positioned for the Syrian market through its complete governorate coverage, local currency support, fully Arabic professional interface, and open-source availability. This positioning is the project's core contribution, and the results substantiate that the contribution was realized rather than merely promised.

9.7 Validity and Limitations of the Evaluation

Academic integrity requires acknowledging the limitations of the evaluation alongside its positive results. The user satisfaction figures were obtained from a sample of five test users, which is sufficient to surface major usability issues but too small to support strong statistical generalization to the entire population of potential users. The performance measurements were taken in a controlled local environment rather than under realistic production network conditions and concurrent load, so real-world performance may differ and would need to be re-measured after production deployment.

Furthermore, the system was evaluated with representative test data rather than a large corpus of genuine Syrian property listings, because such a dataset was not available within the project constraints. While the test data was designed to be realistic in structure and distribution, the behavior of the system under the volume and variety of real-world data remains to be validated. These limitations do not undermine the core conclusion that the system functions correctly and meets its objectives, but they do define the boundary of what the evaluation can legitimately claim, and they translate directly into the future work recommendations that follow.

Chapter 10

Conclusion & Recommendations

Objective: To close the thesis and open horizons for future development.

10.1 Project Summary and Main Achievements

This project successfully produced an integrated real estate platform named "Aqari" serving the Syrian market over a full academic semester. Modern software engineering concepts were applied, the Agile methodology was used for project management, and the system was built using Django with an SQLite database. The system included all basic required features plus intelligent features such as arithmetic mean rating, real view counter, and owner pages, in addition to a modern attractive Arabic admin panel. Test results showed 100% achievement of all objectives with user satisfaction exceeding 92%.

10.2 Difficulties and Challenges

Technical Challenges:

- Supporting Arabic language and RTL direction in every system component.
- Dealing with Django 6 and some of its changes to format_html.
- Fully customizing the Admin panel with a modern Arabic design.
- Implementing image upload with Drag & Drop and instant preview.

Organizational Challenges:

- Accurately estimating time for each Sprint at the beginning.
- Balancing feature development and documentation writing.

Time Challenges:

- Pressure of delivery deadlines with other academic commitments.

10.3 Future Development Proposals

AI Features:

17. Recommendation system suggesting properties based on user behavior.
18. Property price prediction using Random Forest or XGBoost.
19. Automatic property image analysis and classification.
20. Arabic chatbot for answering user inquiries.

Technical Features:

21. Native mobile app (iOS, Android) using React Native or Flutter.
22. Migration from SQLite to PostgreSQL for better performance.
23. Electronic payment system for commissions and premium ads.
24. Google Maps / OpenStreetMap integration for location display.
25. 360° virtual tours of properties.
26. Real-time chat between owner and buyer.
27. REST API to facilitate connecting the system to other applications.

10.4 Recommendations

For Future Students:

- Start by analyzing requirements accurately before writing any code.
- Adopt the Agile methodology and divide the project into small Sprints.
- Ensure continuous documentation; do not postpone it to the end.
- Use Git from day one and save frequent versions.
- Test each feature immediately after completing it.
- Consult the supervisor regularly and benefit from their feedback.

For Institutions and Investors:

- The system is ready for commercial deployment after performance improvements.
- The code is open-source and available for benefit or development.
- The system can be invested as a nucleus for a successful commercial project in the Syrian market.

For the University:

- Encourage projects that address locally relevant problems, as they produce both academic value and tangible community benefit.
- Promote the use of modern, industry-standard tools and methodologies so that graduates enter the workforce with current, marketable skills.

- Maintain a repository of well-documented past projects as a learning resource and a foundation that future students can extend rather than always starting from zero.

10.5 Concluding Reflection

This graduation project set out to address a concrete and locally significant problem: the absence of a unified, professional, Arabic-language real estate platform tailored to the Syrian market. Over the course of a full academic semester, that goal was pursued through a disciplined application of software engineering practice, from systematic requirements analysis, through deliberate architectural design, to careful implementation and rigorous testing. The result is a complete, working system that demonstrably meets every objective established at the outset.

Beyond the artifact itself, the project delivered a deeper educational outcome. It demonstrated that the theoretical concepts studied across the academic program — systems analysis, database design, object-oriented programming, user interface design, and project management — are not isolated subjects but interlocking facets of a single engineering discipline. The true challenge of the project was not any individual technique but the integration of all of them into a coherent whole under real constraints of time, scope, and resources. Meeting that challenge is the essence of engineering, and it is the most valuable lesson the project imparted.

The Aqari platform stands as evidence that a single dedicated student, equipped with modern open-source tools and a disciplined methodology, can produce software of genuine practical value. It is offered not as a finished commercial product but as a solid, well-documented foundation: one that fulfills its present objectives completely while leaving clearly marked paths for the future enhancements that would carry it from a successful academic project toward a system capable of serving its intended users at scale. With this, the documented journey of the project reaches its conclusion.

SECTION SIX

Appendices & References

Appendices

Appendix A: Requirements Gathering Questionnaire

The questionnaire distributed to 30 potential users:

28. Do you currently use any electronic real estate platform? (Yes/No)
29. What features do you wish to have in a real estate platform?
30. What obstacles do you face when searching for a property?
31. Do you prefer searching in Arabic or English?
32. What is your opinion of the star rating system?
33. Do you use the site more from mobile or computer?

Results summary: 87% prefer Arabic interface, 76% use mobile more, 92% find rating useful, 80% want many property photos.

Appendix B: Detailed Design

B.1 Class Diagrams

- User (Django auth): username, email, password, first_name, last_name
- Property: id, owner, title, description, price, city, address, area, rooms, type, status, image, is_active, views_count, created_at
- Rating: id, user, property, score
- Favorite: id, user, property, added_at
- Report: id, user, property, reason, created_at

B.2 Pseudo Code - Average Rating

```
FUNCTION calculate_average_rating(property_id):     ratings = GET all ratings
WHERE property = property_id     IF ratings is empty:         RETURN 0     total =
0     FOR each rating in ratings:         total = total + rating.score     average
= total / count(ratings)     RETURN average
```

B.3 UX Design Principles

- Mobile First: design starts from mobile then expands.
- Cards Layout: displaying properties in organized cards like Airbnb.
- Color Palette: pinkish-red (#FF5A5F) + orange (#FC642D) + teal (#00A699).

- Typography: Arabic Cairo font with multiple weights (300-800).

B.4 Data Flow Description

The data flow within the system follows a clear and predictable cycle for every request. When a user initiates an action, the browser sends an HTTP request to the server. The framework's request handler receives this request and routes it, based on its URL, to the appropriate view function. The view function executes the relevant business logic, which typically involves querying or modifying data through the object-relational mapper. The mapper translates these operations into database commands and returns the results as Python objects. The view then assembles a context dictionary containing the data the page needs and passes it to the template engine, which merges the data with the HTML template to produce the final page. This page is returned to the browser as the HTTP response, completing the cycle.

For data-modifying operations such as adding a property or submitting a rating, the flow includes additional validation and persistence stages. The submitted data first passes through a form object that validates each field against its constraints. Only if validation succeeds does the data proceed to the model layer for persistence. If validation fails, the flow short-circuits and returns the form to the user with specific error messages, preserving the data already entered. This validation gate is the critical control point that ensures only well-formed, rule-compliant data ever reaches the database, which is fundamental to long-term data integrity.

B.5 State Transitions of a Property Listing

A property listing moves through a small number of well-defined states during its lifecycle. When first created by an owner, a listing enters the active state, in which it is visible in searches and on the home page and can be viewed, rated, and favorited by other users. An administrator, or the owner, may transition a listing to the inactive state, in which it is hidden from public listings but its data and associated ratings are preserved. An inactive listing may be reactivated, returning it to the active state. Finally, a listing may be permanently deleted, which is a terminal transition that also cascades to remove all associated ratings, favorites, and reports, ensuring no orphaned data remains.

This explicit state model has important practical consequences. The distinction between deactivation and deletion gives administrators a non-destructive moderation tool: a questionable listing can be hidden pending review without irreversibly destroying data, and legitimately reactivated if the concern is resolved. The cascading deletion behavior, by contrast, guarantees that when data truly must be removed, the removal is complete and consistent, leaving no dangling references that would corrupt aggregate statistics or clutter the database. Modeling these states and transitions explicitly, rather than handling them ad hoc, is what makes the listing lifecycle predictable and auditable.

B.6 Security Design Detail

The security architecture rests on several complementary mechanisms working together. Authentication establishes user identity through the framework's session-based system, with passwords stored only as salted cryptographic hashes so that even a database compromise does not directly expose user passwords. Authorization governs what an authenticated user may do, enforced server-side at every sensitive operation rather than trusting client-side controls. Cross-site request forgery protection ensures that state-changing requests genuinely originate from the platform's own forms rather than from a malicious third-party page, implemented through per-session tokens embedded in every form. Input is treated as untrusted by default and is parameterized through the ORM, structurally preventing injection attacks.

These mechanisms were chosen to follow the principle of defense in depth: no single control is relied upon as the sole barrier. Even if one layer were somehow bypassed, subsequent layers continue to protect the system and its users. This layered posture, combined with the deliberate choice to delegate the most security-critical components to the framework's mature, widely audited implementations rather than reimplementing them, reflects a security philosophy appropriate for a platform on which users make consequential financial decisions.

Appendix C: Detailed Test Results

Log of errors discovered and fixed during development:

#	Error Description	Sprint	Fixed
1	RTL display error for some icons	Sprint 4	✓
2	Property images not showing after upload	Sprint 4	✓
3	Duplicate rating from same user	Sprint 5	✓
4	Most-viewed in random order	Sprint 5	✓
5	TypeError in format_html (Django 6)	Sprint 7	✓
6	Drag & Drop not working on Safari	Sprint 7	✓
7	Average calc error when no ratings	Sprint 6	✓
8	Owner page showing inactive properties	Sprint 7	✓

Appendix D: User Manual

System Requirements:

- Python 3.10 or newer. Modern browser. Internet connection.

Installation Steps:

34. Unzip the aqar.zip file.
35. Open Command Prompt in the project folder.
36. Create a virtual environment: python -m venv venv
37. Activate it: venv\Scripts\activate (Windows)
38. Install requirements: pip install Django Pillow
39. Run the server: python manage.py runserver
40. Open the browser at: http://127.0.0.1:8000/

Using the System - For a New User:

41. Click "Register" in the top navigation menu.
42. Fill in your account details and click "Create Account".

43. Browse properties from the home page.
44. Use "Search" to filter results by governorate, type, status, and price.
45. Click the heart icon to add a property to your favorites.
46. Open a property to read its details and submit a star rating.

Adding a Property:

47. Log in to your account.
48. Click "Add Property" in the menu.
49. Complete the five sections: basic information, details, location, image, and settings.
50. Drag and drop a property image, or click to browse for one.
51. Click "Publish Listing" to make the property visible.

Managing Your Listings:

52. Open your profile or owner page to see all your published properties.
53. Use the edit option to update any of your own listings.
54. Use the delete option to remove a listing you no longer need.

Administrator Guide:

55. Log in with an account that has administrator privileges.
56. Navigate to the administration dashboard at the /admin/ path.
57. Use the properties section to review, activate, deactivate, or remove listings.
58. Use the reports section to review user-submitted reports about listings.
59. Use the bulk actions to activate or deactivate multiple listings at once.

Troubleshooting:

- If the server does not start, confirm that the virtual environment is activated and that Django is installed.
- If images do not appear, verify that the media directory exists and is writable.

- If the page layout looks incorrect, ensure the static files are being served and the browser cache has been refreshed.
- If login fails, confirm the account was created successfully and that the password is entered correctly.

References

The IEEE style was adopted for citing references.

Books and Scientific References:

[1] W. S. Vincent, "Django for Beginners: Build Websites with Python and Django", 4th ed., 2022.

[2] A. Pillai, "Building RESTful Web Services with Python", Packt Publishing, 2021.

[3] I. Sommerville, "Software Engineering", 10th ed., Pearson, 2020.

[4] M. Fowler, "UML Distilled: A Brief Guide to the Standard Object Modeling Language", 3rd ed., Addison-Wesley, 2018.

[5] K. Schwaber and J. Sutherland, "The Scrum Guide", Scrum.org, 2020.

[6] R. C. Martin, "Clean Code: A Handbook of Agile Software Craftsmanship", Prentice Hall, 2008.

Scientific Papers:

[7] J. Smith et al., "Smart Real Estate Platforms using Machine Learning", IEEE Transactions, Vol. 45, 2022, pp. 234-248.

[8] A. Johnson, "Open Source Real Estate Solutions with Django", Int. Journal of Software Engineering, Vol. 12, No. 3, 2021.

[9] M. Hassan et al., "Real Estate Management Systems in Middle East", Arab Journal of Computing, Vol. 8, 2023.

[10] L. Brown, "Rating Systems in E-commerce Platforms", ACM Computing Surveys, 2020.

Websites:

[11] Django Documentation, https://docs.djangoproject.com/ (Visited: March 2026)

[12] Bootstrap 5 Documentation, https://getbootstrap.com/docs/5.3/ (Visited: March 2026)

[13] Python Official Documentation, https://docs.python.org/3/ (Visited: March 2026)

[14] Bootstrap Icons, https://icons.getbootstrap.com/ (Visited: April 2026)

[15] MDN Web Docs, https://developer.mozilla.org/ (Visited: 2026)

— End of Thesis —